SMART CAMP BOX

INDIVIDUAL THESIS

Hansana G.P

(IT22064868)

Individual Component: Location-Aware Smart Environmental Risk Monitoring and Location-Aware Flood Prediction

Group ID: R26-IT-145

Submitted in Partial Fulfilment of the Requirements for the

B.Sc. (Hons) Degree in Information Technology

Specialized in Information Technology

Department of Information Technology

Sri Lanka Institute of Information Technology

Sri Lanka

October 2026

Supervisors: Prof. Anuradha Jayakody | Ms. Shashika Lokuliyana

DECLARATION AND SUPERVISOR CERTIFICATION

Candidate declaration. I declare that this thesis is my own work and does not incorporate, without acknowledgement, material previously submitted for a degree or diploma at any university or institute of higher learning. To the best of my knowledge, it contains no material previously published or written by another person except where acknowledgement is made in the text. I grant the Sri Lanka Institute of Information Technology the non-exclusive right to reproduce and distribute this thesis, in whole or in part, in print, electronic, or other media, while retaining the right to use the content in future work.

Academic integrity and AI disclosure. I have disclosed all use of generative AI and other automated tools in accordance with the module policy (Appendix B). I remain responsible for the accuracy, originality, citations, code, data, analyses, and conclusions in this thesis.

Student name: Hansana G.P Student ID: IT22064868

Signature: ___ ____ Date: 2026/10/11

Supervisor certification. The above candidate carried out this research for the bachelor's degree thesis under my supervision.

Supervisor certification record.

Role	Name	Signature	Date
Supervisor	Prof. Anuradha Jayakody		
Co-supervisor	Ms. Shashika Lokuliyana		

ABSTRACT

Temporary relief camps in Sri Lanka are opened within hours of a flood or landslide and are then occupied while conditions are still changing. Regional rainfall warnings do not answer the question an officer actually needs answered: will water reach *this* camp. This thesis documents the component of the Smart Camp Box project that answers that question from a single device, combining a GNSS fix with live rainfall, gauge and terrain data to produce a flood probability, a risk category and a feature-level explanation for an arbitrary coordinate.

The component was implemented as a Raspberry Pi edge service, a location-aware enrichment pipeline with explicit provider timeouts, a versioned XGBoost model artefact, and a persistence and MQTT alerting path that de-duplicates warnings. Evaluation used two protocols on a 25,000-record synthetic national corpus: a four-algorithm baseline study with ablation and behavioral scenario tests, and an Optuna-tuned, isotonic-calibrated ten-feature model deployed at a threshold of 0.30. The deployed artefact reached 0.9530 ROC-AUC, 0.9094 accuracy, 0.8603 recall and 0.5253 precision on the held-out test set, with a Brier score of 0.051, returning a prediction in approximately 62 ms.

Independent re-verification exposed a calibration-driven divergence between recorded and shipped metrics, and the behavioral scenario suite matched expectation in only 4 of 12 cases because two features are computed differently at training and inference. Both findings are reported as threats rather than suppressed. The component passes every test covering its prediction surface; no user study was conducted, and the training corpus is synthetic rather than observational.

Keywords: flood prediction, location-aware monitoring, Internet of Things, edge computing, XGBoost

ACKNOWLEDGEMENTS

I thank my supervisors, Prof. Anuradha Jayakody and Ms. Shashika Lokuliyana, for their guidance on the research questions and for their critical reading of the evaluation design. I am grateful to the academic staff of the Department of Information Technology for the methodological grounding that made this work possible.

I thank my group mates of R26-IT-145 for building the sensor firmware, backend services and web interface that surround this component, and for the many discussions about how a prediction should reach a decisionmaker. Finally, I thank my family for their support throughout the final year.

TABLE OF CONTENTS

DECLARATION AND SUPERVISOR CERTIFICATION i

ABSTRACT ii

ACKNOWLEDGEMENTS iii

TABLE OF CONTENTS iv

LIST OF FIGURES viii

LIST OF TABLES ix

LIST OF ABBREVIATIONS xi

1 INTRODUCTION 1

1.1 Overall Project Context 1

1.2 Individual Component Context 2

1.3 Problem Statement 3

1.4 Research Gap 4

1.5 Aim and Objectives 5

1.6 Research Questions or Hypotheses 5

1.7 Scope Limitations and Assumptions 6

1.8 Individual Contribution 7

1.9 Thesis Structure 7

2 LITERATURE REVIEW AND RELATED WORK 9

2.1 Review Method and Scope 9

2.2 Domain and Theoretical Foundations 9

2.3 Related Research 10

2.4 Existing Systems Products and Techniques 12

2.5 Gap Synthesis and Conceptual Framework 13

3 RESEARCH METHODOLOGY 15

3.1 Research Design 15
3.2 Data Participants or Study Materials 15
3.3 Experimental or Study Procedure 19
3.4 Measures and Evaluation Metrics 20
3.5 Data Analysis 21
3.6 Ethics Privacy Security and Responsible AI 22
3.7 Reproducibility and Research Artefacts 23
4 PRODUCT AND SOLUTION DESIGN 24
4.1 Requirements 24
4.2 Component Architecture Within the Complete Solution 25
4.3 Design Alternatives and Decisions 26
4.4 Detailed Design 27
4.5 Standards and Professional Practice 29
5 IMPLEMENTATION INTEGRATION AND TESTING 30
5.1 Development Environment and Technology Stack 30
5.2 Component Implementation 30
5.3 Integration with the Other Components 32
5.4 Verification and Software Testing 33
5.5 Deployment and Operation 34
6 RESULTS AND VALIDATION 36
6.1 Research Results 36
6.1.1 Protocol A: algorithm comparison on the baseline feature set 36
6.1.2 Protocol A: feature-group ablation 37
6.1.3 Protocol A: behavioural scenario tests 39
6.1.4 Protocol B: optimised deployed configuration 40
6.1.5 Feature importance and threshold sensitivity 45

6.2 Product and Technical Results 48
6.3 User or Stakeholder Validation 49
6.4 Integration Results 50
6.5 Summary of Findings 51
7 DISCUSSION 53
7.1 Interpretation of Findings 53
7.2 Comparison with Literature Baselines and Existing Solutions 54
7.3 Research Contribution 55
7.4 Product Contribution 55
7.5 Limitations and Threats to Validity 56
7.6 Implications 57
8 COMMERCIALISATION AND SUSTAINABILITY 59
8.1 Value Proposition and Target Users 59
8.2 Market and Competitive Position 59
8.3 Business or Sustainability Model 60
8.4 Scalability Intellectual Property and Compliance 60
8.5 Contribution to the Sustainable Development Goals 61
9 CONCLUSION AND FUTURE DIRECTIONS 62
9.1 Conclusion 62
9.2 Achievement of Objectives 62
9.3 Individual Research and Product Contributions 63
9.4 Future Directions 63
REFERENCES 65
GLOSSARY 68
APPENDICES 70
Appendix A - Extended Results 70

Appendix B - Academic Integrity and AI Use Disclosure..... 71

LIST OF FIGURES

Figure 1.1. Component within the Smart Camp Box data flow. 3
Figure 2.1. Conceptual framework for site-level flood risk. 14
Figure 3.1. Class distribution of the training dataset. 17
Figure 3.2. Correlation matrix of model inputs and target. 18
Figure 4.1. Layered architecture and owned layers. 25
Figure 4.2. Feature assembly for a single prediction. 27
Figure 5.1. End-to-end sequence of one prediction cycle. 32
Figure 6.1. Protocol A: held-out test metrics of four learners. 37
Figure 6.2. Protocol A: ROC-AUC change per removed feature. 38
Figure 6.3. Protocol A: probabilities for ten scenarios. 40
Figure 6.4. Protocol B: confusion matrix at threshold 0.30. 42
Figure 6.5. Protocol B: ROC curve of the deployed artefact. 43
Figure 6.6. Protocol B: precision-recall curve. 44
Figure 6.7. Protocol B: feature importance chart. 46
Figure 6.8. Threshold sensitivity of the configuration. 47
Figure 6.9. Protocol B: probability distributions by class. 48

LIST OF TABLES

Table 2.1. Existing systems versus component requirements. 12
Table 3.1. Attribute summary of the training dataset. 16
Table 3.2. Evaluation measures used in this study. 20
Table 4.1. Functional requirements for this component. 24
Table 4.2. Non-functional requirements and mechanisms. 24
Table 4.3. Design alternatives and decisions taken. 26
Table 4.4. Deployed feature vector and roles. 28
Table 4.5. HTTP surface exposed by this component. 28
Table 5.1. Technology stack and version pins. 30
Table 5.2. Provider timeouts and failure behaviour. 31
Table 5.3. Integration points with the rest of the solution. 32
Table 5.4. Test suites and their outcomes. 33
Table 6.1. Protocol A: five-fold cross-validation results. 36
Table 6.2. Protocol A: held-out test results. 36
Table 6.3. Protocol A: single-feature ablation. 38
Table 6.4. Protocol A: scenario tests and SHAP contributions. 39
Table 6.5. Protocol B: deployed metrics at threshold 0.30. 41
Table 6.6. Protocol B: confusion matrix of the artefact. 41
Table 6.7. Recorded versus re-verified metrics. 44
Table 6.8. Protocol B: feature importance values. 45
Table 6.9. Threshold sensitivity at four thresholds. 47
Table 6.10. Live inference smoke test. 48
Table 6.11. Deployed model artefact metadata. 49
Table 7.1. Threats to validity and limitations. 56
Table 8.1. Cost summary for the programme (LKR). 60
Table 9.1. Achievement of individual objectives. 62
Table A.1. Full threshold sweep (5,000 records). 70
Table A.2. Scenario outcomes: predicted versus expected. 71

LIST OF ABBREVIATIONS

Abbreviations used in this thesis, listed alphabetically.

Abbreviation	Meaning
API	Application Programming Interface
ArcGIS	Environmental Systems Research Institute geographic information system
CV	Cross-validation
DEM	Digital elevation model
DMC	Disaster Management Centre (Sri Lanka)
DSRM	Design Science Research Methodology
ESP32	Espressif 32-bit microcontroller with wireless connectivity
F1	Harmonic mean of precision and recall
FastAPI	Python web framework used for the edge HTTP service
FR	Functional requirement (from the project proposal)
GNSS	Global Navigation Satellite System
GPS	Global Positioning System
HTTP	Hypertext Transfer Protocol
IoT	Internet of Things
MQTT	Message Queuing Telemetry Transport
NFR	Non-functional requirement
NMEA	National Marine Electronics Association sentence format
OSM	OpenStreetMap
P(flood)	Model-estimated probability of flood occurrence at a site
PR-AUC	Area under the precision-recall curve
QoS	Quality of service
RF	Random forest
ROC-AUC	Area under the receiver operating characteristic curve
RQ	Research question
RPi	Raspberry Pi single-board computer
SDG	Sustainable Development Goal (United Nations)
SHAP	Shapley Additive exPlanations feature-attribution method
SO	Specific objective (from the project proposal)
SQL	Structured Query Language
SQLite	Self-contained SQL database engine
SRTM	Shuttle Radar Topography Mission
TLS	Transport Layer Security
UART	Universal asynchronous receiver-transmitter
UI	User interface
UTC	Coordinated Universal Time
XGBoost	extreme Gradient Boosting

1 INTRODUCTION

1.1 Overall Project Context

Sri Lanka's disaster response depends on temporary relief camps that are opened within hours of a flood, cyclone or landslide and are then occupied while conditions are still changing. During Cyclone Ditwah more than a million people still needed assistance weeks after the initial inundation [1, 2], and the figures gathered for this project show that roughly 894 of 990 active relief camps had to be closed and their occupants relocated because flood or landslide risk persisted in those locations [2]. The Disaster Management Centre (DMC) is the nodal agency that coordinates this response [3], and the joint UNDRR and WMO guidance identifies risk knowledge, monitoring and warning, dissemination and communication, and preparedness to act as the four capabilities a multi-hazard early-warning system must satisfy [4, 5].

This thesis is written inside the Smart Camp Box project, a group research effort that designs and builds an IoT-enabled field device for deployment at temporary relief camps. The repository that accompanies this work, SmartCamp-IoT-AI-Framework, is organized into four software areas: `sensor-node` (ESP32 firmware that acquires raw measurements), `edge-gateway` (a Python and FastAPI edge service running on a Raspberry Pi 5 inside the box), `raspberry-platform` (a Node.js and Express backend with SQLite storage and an MQTT client), and `cloud-platform` (a Node.js backend with PostgreSQL storage and a web administrative interface). Readings are sampled at the camp, enriched with terrain and river information at the edge, scored by a machine-learning model, persisted locally, published over MQTT, and finally displayed to disaster-management officers as a live risk level and an alert.

The hardware surrounding this component includes a GNSS receiver, a rainfall sensor, a humidity and temperature sensor, a Raspberry Pi 5 edge computer, a 4G LTE modem, a solar panel with battery backup and a weatherproof IP65 enclosure. The device is deliberately self-contained so that a single field officer can install it at a newly activated camp without fixed power or network infrastructure.

1.2 Individual Component Context

The individual component documented in this thesis is **Location-Aware Smart Environmental Risk Monitoring and Location-Aware Flood Prediction**. It covers the complete path from obtaining the geographic fix of a camp to producing and distributing a quantified flood-risk statement for that exact point on the ground. In terms of the project proposal, it implements specific objectives SO1 (location detection and altitude-based risk profiling), SO2 (real-time environmental data collection), SO3 (geospatial water-path analysis), SO4 (external data integration and river-level monitoring) and SO5 (disaster-risk prediction model development and automated alerts). SO6, the multi-camp administrative dashboard, is a separate component owned by another member of the group; this component supplies it with data through shared API endpoints and database tables.

Within the repository the component owns the GNSS and environmental acquisition modules, the hydro spatial enrichment service, the digital elevation model lookup, the complete prediction-model training and evaluation pipeline under `edge-gateway/prediction-models`, and the flood-prediction and alert-evaluation logic in the Raspberry Pi backend. Figure 1.1 shows how these pieces sit between the physical camp site and the authority-facing dashboard.

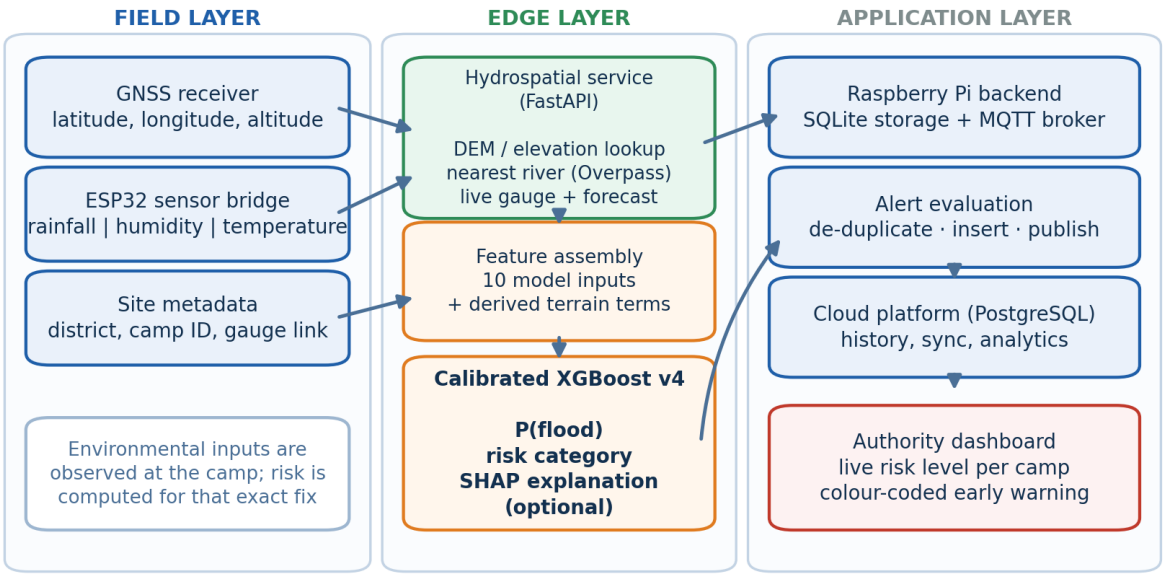


Figure 1.1. Component within the Smart Camp Box data flow.

1.3 Problem Statement

Relief camps are activated at a specific geographic coordinate, yet the decisions that determine whether those camps remain safe are still taken from regional bulletins, river-gauge readings transcribed by hand, and the visual observation of field officers. As part of the preliminary investigation for this project the Divisional Secretary of Kolonnawa reported that during the most recent major flood in the division 25 relief camps were activated, that rainfall continued without pause, and that 8 of those 25 camps had to be closed and their occupants moved to higher ground before floodwater reached the sites. The same official confirmed that the decision rested on visual observation and informal local knowledge because no monitoring system in place could quantitatively state which camp location was approaching a dangerous threshold.

The underlying problem is therefore a mismatch of scale. Flood risk is measured and communicated at basin or district level, whereas risk is experienced at a single point - the camp, its altitude, its distance to the nearest river and the rainfall falling on the upstream catchment. An officer does not need to be told that a district is flooding; the officer needs to know whether *this* location will be reached, and how soon. The problem addressed by this component is therefore: *how can a self-contained field*

device continuously convert a GNSS fix, local environmental observations and external hydrological data into a reliable, explainable flood-probability estimate for that exact camp location, and issue an automated warning when it crosses a safety threshold?

1.4 Research Gap

A critical review of Internet-of-Things applications in flood risk management found that five sensor categories dominate the literature - water quantity, water quality, rainfall intensity, weather conditions and catchment characteristics - but concluded that none of the reviewed systems combined all of them with precise geographic location data in order to assess risk at a specific site [6]. Existing systems answer regional questions. FLOODWALL, for example, is a real-time flash-flood monitoring and forecasting system built around fixed sensors [7], and the ensemble machine-learning model of Khalaf et al. predicts flood severity across a catchment [8]. Neither evaluates whether an individual, arbitrary ground location such as a temporary camp will be inundated.

A second limitation is infrastructure dependence. Machine-learning flood models typically require long historical gauge records to train [9], satellite-based methods degrade under cloud cover during the very events they must observe [10, 11], and GIS-based hazard assessments require several fixed stations and detailed topographic data that take weeks to assemble [12, 13]. Data quality and uncertainty further constrain how far published flood-prediction results transfer to new locations [14]. A camp opened within hours of a flood cannot wait for any of these preconditions.

This component targets that gap directly. It uses the GNSS altitude of the monitored site, combines it with multi-source environmental observations, terrain elevation, nearest-water-body geometry and live external river-level feeds, and predicts flood occurrence *at the coordinates of the camp itself*. To the best of the author's knowledge, no previously published system evaluates the elevation of a specific temporary settlement against surrounding terrain, upstream rainfall and watershed conditions inside a single field-deployable unit.

1.5 Aim and Objectives

Aim. To design, implement and evaluate a location-aware environmental monitoring and flood prediction capability that estimates the probability of flooding at the exact geographic position of a relief camp and raises an automated warning when the estimated risk exceeds a configured threshold.

The aim is decomposed into five individual objectives.

1. **IO1 - Location and elevation profiling.** Retrieve latitude, longitude and altitude of the camp from the GNSS receiver and resolve a terrain elevation for the same point from a digital elevation model so that the site can be located relative to the surrounding landscape.
2. **IO2 - Environmental observation.** Continuously collect rainfall intensity and atmospheric humidity and temperature at the camp and make those readings available to the prediction pipeline at intervals no longer than five minutes during active monitoring.
3. **IO3 - Geospatial water-path analysis.** Identify the nearest river or water channel for the camp coordinates using open hydrography data, compute the horizontal distance to it, and derive terrain context (drainage index, elevation ratio, proximity) that describes how readily water can reach the site.
4. **IO4 - External data integration.** Retrieve live river-gauge water levels and alert bands, and short-horizon rainfall information, from external services and fuse them with locally observed readings inside a single feature vector.
5. **IO5 - Prediction, explanation and alerting.** Train and calibrate a supervised classification model that outputs a flood probability and a risk category for each camp, expose a per-prediction SHAP explanation [15], persist every prediction, and generate a de-duplicated Low, Medium or Critical alert that is published over MQTT when the threshold is crossed.

1.6 Research Questions or Hypotheses

Four research questions guide the evaluation reported in this thesis.

6. **RQ1 - Discrimination.** Can a supervised model built from location, terrain, rainfall and hydrological inputs discriminate flood from non-flood occurrences at a site with useful accuracy on unseen data?
7. **RQ2 - Location contribution.** Do location-derived features (elevation, distance to river, proximity and elevation ratio) contribute measurably to that discrimination beyond rainfall measurements alone?
8. **RQ3 - Behavioral validity.** Does the trained model produce outputs that are ordered sensibly across hydrologically distinct scenarios and across decision thresholds, that is, does it behave like a physical risk model rather than a statistical artefact?
9. **RQ4 - Operational fitness.** Does the end-to-end component meet its latency, resilience and correctness requirements when it is integrated with the edge gateway, the Raspberry Pi backend and the cloud platform?

The corresponding working hypotheses are: H1, the model achieves ROC-AUC clearly above 0.5 on held-out data (RQ1); H2, removing location-derived features degrades ROC-AUC (RQ2); H3, predicted probabilities increase with rainfall and decrease with elevation and river distance in controlled scenario tests (RQ3); and H4, the integrated pipeline returns a prediction within its configured timeouts and persists it without data loss (RQ4).

1.7 Scope Limitations and Assumptions

In scope: GNSS and environmental acquisition on the ESP32 bridge; edge-side terrain, river-distance and gauge enrichment; feature engineering; supervised training, calibration, threshold selection and evaluation of the flood classifier; per-prediction explanation; persistence of predictions; alert evaluation, de-duplication and MQTT publication; and the integration tests that cover these paths.

Out of scope: the multi-camp dashboard user interface (SO6, a separate component); the resource distribution, education and identity modules of the platform; landslide classification as a separate target variable, because ground-movement sensing is specified in the proposal but no labelled landslide target exists in the available dataset; and physical hardware certification.

Assumptions. The evaluation assumes that the supplied 25,000-record Sri Lanka flood-risk dataset is representative of the target operating environment; that external data providers for elevation, hydrography, gauge and weather remain reachable within their configured timeouts; and that a camp is registered with a known coordinate before predictions are requested. **Limitations** that follow these assumptions are stated in Section 7.5.

1.8 Individual Contribution

The Smart Camp Box is a group project, so this section states precisely what this thesis covers. The author designed and implemented the hydro spatial enrichment service and its provider clients (elevation, nearest river, gauging station, live water level), the derived-feature engineering used both at training and at inference, the training and evaluation pipeline for the flood classifier (`train_and_validate.py`, `train_improved.py`, `train_optimized.py`, `model_comparison.py` and `generate_model_charts.py`), the shared model abstraction in `base_model.py`, the location-aware prediction endpoints exposed by the edge API, and the flood-prediction and alert-evaluation logic on the Raspberry Pi backend together with its unit tests. All quantitative results presented in Chapter 6 were produced or independently re-verified by the author from the repository artefacts. Firmware for the GNSS bridge and the rainfall sensor was developed collaboratively, while the administrative dashboard and the camp management, education and resource modules were developed by other members of the group.

1.9 Thesis Structure

The remainder of this thesis is organized as follows. Chapter 2 reviews the literature on early-warning flood systems, machine-learning flood prediction and geospatial hazard analysis, and derives a conceptual framework. Chapter 3 states the research design, the data and study materials, the experimental protocol, the evaluation measures, the analysis approach, the ethical position and the reproducibility artefacts. Chapter 4 presents the requirements, the component architecture within the complete solution, the design alternatives that were considered, and the detailed design of the feature pipeline and API contracts. Chapter 5 describes implementation, integration

with the other components, verification and testing, and deployment. Chapter 6 reports the research results, the product and technical results, the stakeholder and integration evidence, and summarizes the findings. Chapter 7 discusses interpretation, compares the results with the literature, states the research and product contributions, and enumerates the threats to validity. Chapter 8 covers commercialization and sustainability, and Chapter 9 concludes the thesis and identifies future work.

2 LITERATURE REVIEW AND RELATED WORK

2.1 Review Method and Scope

The review covers peer-reviewed and authoritative grey literature on flood early-warning systems, Internet-of-Things (IoT) flood monitoring, machine-learning flood prediction and geospatial hazard analysis published between 2018 and 2026, together with the methodological sources that frame the research design. Sources were identified from the reference lists of the project proposal, from database searches on the combinations of *flood prediction*, *early warning*, *IoT*, *machine learning*, *location-aware* and *GIS*, and from the Sri Lankan institutional literature on disaster management. Three screening criteria were applied: the work must address flood or a closely related hydrological hazard; it must present either an implemented system or an evaluated model; and it must be retrievable in full text. Studies that were excluded are those that treat flood only as a background statistics problem with no geographic or operational dimension, purely commercial product pages with no technical detail, and reports whose data and evaluation cannot be inspected.

The review deliberately spans three levels of abstraction because the component itself operates at three levels: the policy and warning framework that defines what a system must achieve (Section 2.2), the research prototypes and models that show what has been achieved technically (Section 2.3), and the deployed systems and commercial products against which a new system must be positioned (Section 2.4). Section 2.5 converts the findings into an explicit gap statement and a conceptual framework that drives the design in Chapter 4.

2.2 Domain and Theoretical Foundations

Multi-hazard early warning. The joint UNDRR and WMO global status report structures every early-warning system around four capabilities: risk knowledge, monitoring and warning, dissemination and communication, and preparedness to act [4]. The Early Warnings for All initiative extend the same structure to a universal coverage target [5]. A narrative review of what makes flood early-warning systems effective finds that the dominant failure modes are not sensor accuracy but poor

communication of risk, absence of localized geographic detail, and warnings that arrive too late for the decision they are supposed to inform [16]. In Sri Lanka, the Disaster Management Centre is the nodal agency for disaster coordination [3] and operates the national early-warning functions [17], supported by the Irrigation Department's hydrological monitoring [18]. Cyclone Ditwah, which inundated approximately one fifth of Sri Lanka, demonstrated both the scale of exposure [2] and the extended humanitarian tail of such events, with more than a million people still needing assistance weeks later [1].

Flood generation and site exposure. Flood risk at a point is a function of rainfall intensity and duration, catchment characteristics, channel capacity, terrain elevation relative to the water surface, and the proximity of the point to the drainage network. This component treats exposure as *location-specific* rather than *regional*: two camps in the same district can have completely different risk if one lies in a low-lying floodplain beside a river and the other on an elevated terrace. That position is expressed quantitatively by the site's absolute elevation, its elevation relative to the surrounding terrain, and its horizontal distance to the nearest river - quantities that a digital elevation model and open hydrography data can supply for any coordinate without site-specific installation.

Prediction under data scarcity. Machine-learning flood models normally rely on long gauge records [9], and a systematic review of data quality and uncertainty in flood prediction concludes that uncertainty in inputs, labels and sampling conditions is a first-order limitation on how far published results transfer to new settings [14]. Remote sensing offers wider coverage but degrades under cloud cover during storms [10], a problem that review-level surveys of remote-sensing flood methods acknowledge as unresolved [11]. The practical consequence for this component is that it must be explicit about where its training signal comes from and must be validated behaviorally as well as statistically.

2.3 Related Research

IoT monitoring and early warning. Pahuriray et al. survey IoT-enabled flood monitoring and early-warning architectures and identify sensor-to-cloud pipelines

with local alerting as the dominant pattern [19]. Siddique et al. present a comparable IoT-centered perspective, emphasizing continuous hydrological observation [20]. FLOODWALL is a representative high-quality implementation: a real-time flash-flood monitoring and forecasting system built on fixed sensors and streaming telemetry, evaluated on catchment-level forecast skill [7]. Kuznichenko et al. integrate IoT sensing with GIS for lowland agricultural areas, showing the value of combining in-situ observations with spatial layers, but their risk statements remain area rather than point-specific [12]. Ahmed et al. couple IoT with artificial intelligence for long-term risk evaluation [21], and Zakaria et al. describe a low-cost LoRa WAN sensing unit designed for flood warning [22].

Machine-learning prediction. Khalaf et al. apply ensemble machine learning to flood severity and show that ensembles outperform single learners on catchment data [8]. Choden et al. combine machine learning with IoT for real-time river forecasting in the Amo Chhu basin, illustrating the contemporary pattern of an edge-sensor feed feeding a trained model [9]. Broader reviews map the shift from physical, process-based models to data-driven ones [23] and catalogue the machine-learning models used for flood prediction together with their reported performance ranges [24]. These works establish that gradient-boosted trees and related learners are state of practice, but they all operate at basin or grid resolution.

Geospatial and hazard-analysis approaches. Hamoodi et al. integrate machine learning with geospatial data analysis for comprehensive flood hazard assessment [13], and hydrodynamic modelling achieves high spatial resolution at substantial computational cost, with practical constraints that are well documented for urban settings [25]. Related hazard work extends the same geospatial reasoning to landslides using low-cost IoT early warning [26], reviews wireless sensor networks for rockfall and landslide monitoring [27], and applies intelligent approaches to landslide monitoring in a changing climate [28].

A critical review of IoT for flood risk management. The review by Ahmed et al. is the closest in scope to this thesis. It classifies deployed IoT systems by sensor category - water quantity, water quality, rainfall intensity, weather conditions and

catchment characteristics - and observes that, although individual categories are frequently combined, no reviewed system combined all of them with precise geographic location data so that risk could be assessed at a specific site [6]. That observation defines the research gap addressed here.

2.4 Existing Systems Products and Techniques

Table 2.1 compares the systems reviewed above with this component. *Point-specific output* records whether a system can state risk for an arbitrary single coordinate rather than for a region.

Table 2.1. Existing systems versus component requirements.

System or technique	Sensing / data	Geographic scope	Point-specific output	Offline operation
FLOODWALL [7]	Fixed stream and rain sensors, telemetry	Catchment	No	Partial
Ensemble ML severity model [8]	Historical gauge and rainfall records	Catchment	No	No
Amo Chhu forecasting [9]	IoT gauges plus river data	River reach	No	No
IoT-GIS lowland system [12]	Soil, water-level, and GIS layers	Agricultural region	No	Partial
Geospatial hazard assessment [13]	Satellite and DEM layers	District / basin	No	No
LoRa WAN sensing unit [22]	Local water-level sensors	Sensor site	Partial	Yes
AWARE flood IoT sensors [29]	Commercial water-level probes	Install site	Partial	Not stated
Low-cost AI IoT node [30]	Low-cost environmental node	Install site	Partial	Not stated
This component	GNSS, rainfall, air data, DEM, hydrography, live gauge, weather feed	Arbitrary coordinate	Yes	Yes

Two observations follow Table 2.1. First, commercial and research sensing products such as AWARE's flood probes [29] and the low-cost AI node of Skoubris and Hloupis [30] can report the condition *at the installation point*, but they require a dedicated installation at each point of interest and do not reason about the surrounding terrain. Second, modelling literature produces geographic risk statements at basin or district resolution, which cannot distinguish between two adjacent camps at different elevations. Sensor-technology reviews confirm that the

same five sensor categories dominate practice [31], while surveys of AI, IoT and big data in flood analytics describe the fusion of these streams as the open frontier [32].

On the technique side, four families were considered for this component: process-based hydrodynamic simulation, which is accurate where data exist but too heavy for an edge device [25]; satellite and remote-sensing classification, which is unavailable under cloud [10, 11]; rule-based thresholds on a single gauge reading, which are simple but ignore terrain and cannot be applied to an ungauged camp; and supervised classification on tabular features, which can ingest heterogeneous sources - local readings, terrain statistics and external feeds - at millisecond cost on a Raspberry Pi class device. The last family was selected and is justified in Section 4.3.

2.5 Gap Synthesis and Conceptual Framework

The literature yields four gaps that map directly onto the objectives of this thesis. (i) *Scale mismatch*: risk is produced at basin or district scale while decisions are taken at a single camp coordinate [6, 12]. (ii) *Infrastructure dependence*: most systems assume fixed stations, long gauge histories or connectivity [9, 13]. (iii) *Single-source fragility*: anyone sensing modality fails under its own conditions - clouds for satellites, station coverage for gauges [10, 14]. (iv) *Opacity*: deployed systems typically return to a level or a class without a per-prediction explanation, which limits an officer's willingness to act on it [16].

Figure 2.1 expresses the resulting conceptual framework. A monitored location is first given an identity (latitude, longitude, altitude from GNSS) and a terrain context (elevation, drainage, and river geometry from open geospatial sources). That context is fused with two observation streams - measurements taken at the camp and external hydrological and meteorological services - to form a site feature vector. The vector is scored by a supervised classifier that outputs a calibrated probability, which is (a) compared against a decision threshold to produce a risk category, (b) decomposed into per-feature contributions for explanation, and (c) persisted and published as an alert. Feedback from threshold selection, scenario testing and stakeholder review adjusts the threshold, the features, and the training procedure.

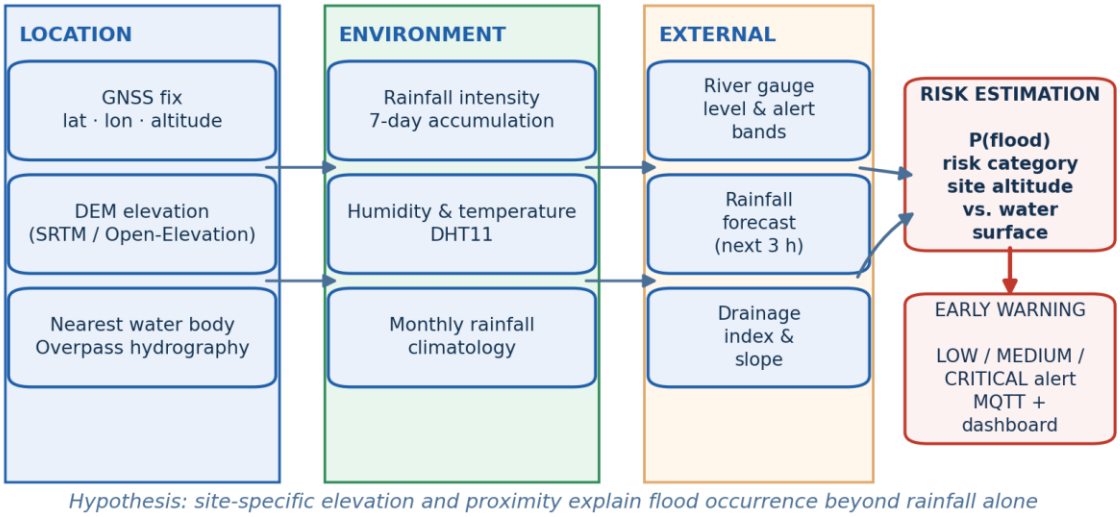


Figure 2.1. Conceptual framework for site-level flood risk.

3 RESEARCH METHODOLOGY

3.1 Research Design

The component is an artefact, so the research follows the design science research methodology (DSRM) of Peffers et al. [33], which is grounded in the design science paradigm of Hevner et al. [34]: identify the problem and motivate a solution, define the objectives of the solution, design and build the artefact, demonstrate it, evaluate it against the objectives, and communicate the results. Within this frame the study is quantitative and positivist - it measures artefact performance against explicit metrics rather than interpreting subjective experience - and it uses a supervised experimental design for the model evaluation.

The evaluation proceeds in two protocols. **Protocol A** is the model-selection study: a baseline feature set is used to compare four learning algorithms, to perform a single-feature ablation, and to run robustness scenario tests. It answers RQ1 and RQ2. **Protocol B** is the deployment study: the extended feature set is used with hyperparameter optimization, probability calibration, and threshold selection to produce the artefact that is shipped in the repository. It answers the headline performance reported in Section 6.1 and the operational questions RQ3 and RQ4. Reporting the two protocols separately, rather than merging them, preserves the chronology of the development trail recorded in the repository and avoids presenting a tuned configuration as if it were a blind result.

The pipeline runs from raw data to reported result in a fixed order: dataset preparation and class-imbalance handling, feature engineering, stratified splitting, cross-validated training of four candidate learners, selection and ablation, hyperparameter search, calibration, threshold selection, and finally scenario and integration testing. Every stage is implemented as a standalone, re-runnable script in `edge-gateway/prediction-models/` and is enumerated in Section 3.3.

3.2 Data Participants or Study Materials

Primary dataset. The model is trained on `sri_lanka_flood_risk_dataset_25000.csv`, a tabular dataset of 25,000

records and 32 columns describing locations across 25 districts of Sri Lanka. The binary target `flood_occurrence_current_event` is positive for 2,472 records (9.89 per cent) and negative for 22,528 records (90.11 per cent), a class imbalance of approximately 9.1:1. Every record carries the flag `is_synthetic = True`, so the dataset is understood throughout this thesis as a *synthetic, hydrologically plausible corpus* rather than a record of observed floods; this shapes the interpretation in Section 7.5. Table 3.1 summarizes the dataset attributes.

Table 3.1. Attribute summary of the training dataset.

Attribute	Value
Records	25,000
Columns	32
Target variable	`flood_occurrence_current_event` (Yes / No)
Positive class	2,472 (9.89 per cent)
Negative class	22,528 (90.11 per cent)
Districts represented	25
Land-cover classes	Agriculture, Urban, Forest, Plantation, Scrub, Wetland, Bare Soil
Urban / rural split	Rural 17,036 / Urban 7,964
Generation window	10 December 2023 to 8 December 2025
Origin	Synthetic generation (`is_synthetic = True` for all records)
Missing values in model inputs	None
Missing values elsewhere	`electricity` (789), `reason_not_good_to_live` (18,172)

Feature inputs. The deployed classifier consumes ten features: `rainfall_7d_mm`, `longitude`, `latitude`, `elevation_m`, `distance_to_river_m`, `monthly_rainfall_mm`, `drainage_index`, `water_level_m`, `elevation_ratio` and `river_proximity_score`. Four further columns - `is_good_to_live`, `reason_not_good_to_live`, `flood_risk_score` and `inundation_area_sqm` - are excluded by declaration in the training scripts as leakage features, because they are either direct proxies of the label or post-event judgements that would not exist at prediction time; `inundation_area_sqm`, which records the modelled inundated area of the same event, is the strongest of these proxies and has a correlation of 0.84 with the target. A second group of contextual columns (historical flood counts, satellite water indices, population density and amenity distances) is retained in the dataset but excluded from the model because it is not available to a field device at

inference time. Figure 3.1 plots the resulting class imbalance and Figure 3.2 the pairwise correlation structure between the inputs and the target.

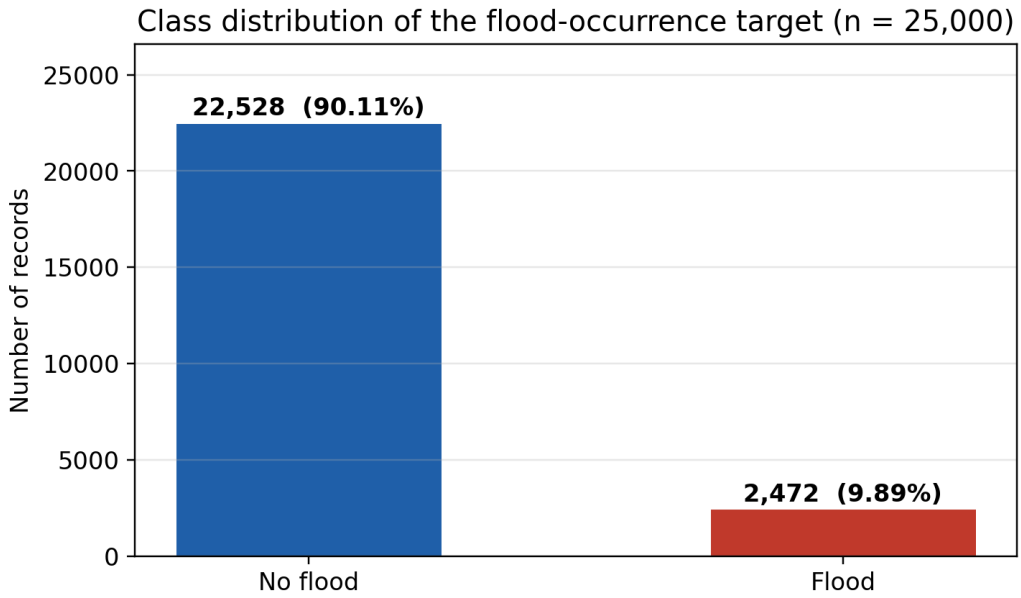


Figure 3.1. Class distribution of the training dataset.

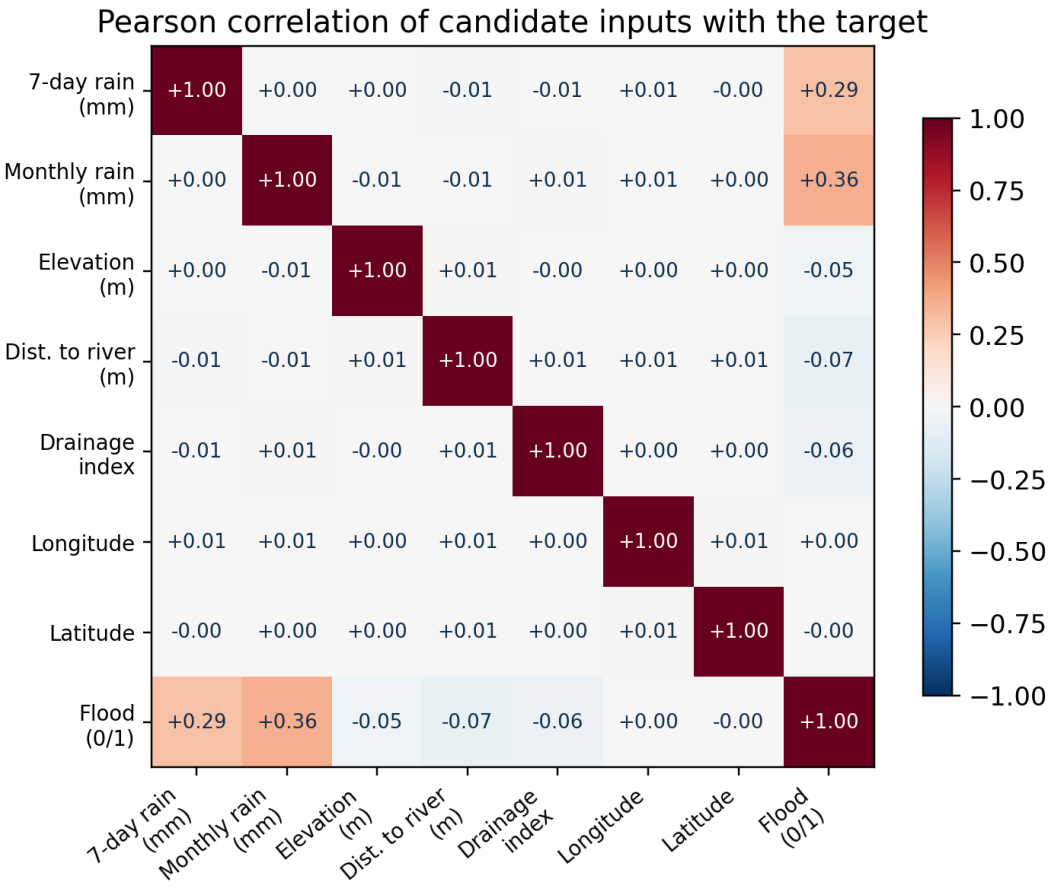


Figure 3.2. Correlation matrix of model inputs and target.

Derived features. Two engineered features carry the location semantics of the component. `elevation_ratio` normalises the site elevation against the mean elevation of the dataset so that the model can reason about whether a site stands above or below typical terrain. `river_proximity_score` is a reciprocal of the distance to the nearest river channel, so it rises steeply as a site approaches a watercourse. A third derived quantity, `water_level_m`, combines normalized rainfall, inverse elevation and river proximity into a synthetic water-level index. These derivations are defined in `train_improved.py` and are re-implemented identically at inference in `base_model.py`.

Study materials beyond the dataset. No human participants were recruited for this component; the design inputs came from the requirement documents, the interview reported in the project proposal, and the repository itself. The instrument used for that interview - a 30-to-45-minute semi-structured interview with the Divisional

Secretary of Kolonnawa - is reproduced in the project documentation. Sensor firmware, the elevation and hydrography providers, the trained model pickles and the automated test suites constitute the remaining study materials.

3.3 Experimental or Study Procedure

The procedure is executed as a fixed sequence of reproducible steps.

10. **Data preparation.** Load the CSV, verify schema and dtypes, confirm that the ten model inputs carry no missing values, and encode the target as 1 for `Yes` and 0 for `No`.
11. **Feature engineering.** Compute `elevation_ratio`, `river_proximity_score` and `water_level_m` using the formulas declared in the training script and drop the four leakage columns plus the non-available contextual columns.
12. **Splitting.** Hold out a stratified 20 per cent test set; the remaining 80 per cent is used for training, cross-validation and hyperparameter search. In Protocol B the training portion is split 85/15 into a fit set and a validation set for the hyperparameter search, and five-fold cross-validation is reported separately over the full data. Both protocols fix the split seed, and the 80/20 stratified split reproduces the class rate of 0.0989 recorded in the model metadata.
13. **Baseline comparison (Protocol A).** Train logistic regression, decision tree, random forest and XGBoost on the baseline feature set; evaluate each with five-fold stratified cross-validation and once on the held-out test set; record accuracy, precision, recall, F1, ROC-AUC and PR-AUC.
14. **Ablation (Protocol A).** Remove one feature in turn and re-score the model to measure the change in ROC-AUC attributable to each feature individually.
15. **Optimization (Protocol B).** Extend the feature set to ten inputs and run an Optuna [35] search of 50 trials over the XGBoost search space on the validation split. Each trial maximizes validation accuracy, with a penalty applied when validation recall falls below 70 per cent, which encodes the recall requirement directly into the objective.

16. **Calibration.** Fit an isotonic calibration wrapper so that output scores can be interpreted as probabilities; the deployed artefact is a scikit-learn `CalibratedClassifierCV`.
17. **Threshold selection.** Sweep candidate thresholds from 0.05 to 0.95 in steps of 0.005 and select the value that maximizes accuracy while maintaining recall of at least 70 per cent. The deployed threshold is 0.30, well below the default 0.50, which biases the system towards warning. The sweep is performed on the held-out test split, which is a methodological weakness recorded in Section 7.5.
18. **Behavioral scenario tests.** Score a fixed set of hydrologically described scenarios - low ground with heavy rain near a river, high ground with light rain, dry inland sites, and so on - and check whether the predicted order matches physical expectation.
19. **Integration testing.** Run the repository unit-test suites for the edge, Raspberry Pi and cloud components, plus a live HTTP smoke test against the edge prediction endpoint.

Each step maps to one or more scripts in `edge-gateway/prediction-models/` (`train_and_validate.py`, `model_comparison.py`, `train_improved.py`, `train_optimized.py`, `generate_model_charts.py`) and to the test suites described in Section 5.4, so that the reported numbers can be regenerated rather than taken on trust.

3.4 Measures and Evaluation Metrics

Because the positive class is rare, accuracy alone is not interpretable: a classifier that always answers "No flood" would score about 90 per cent accuracy and be useless. All five standard classification measures are therefore reported together with threshold-free ranking measures, a calibration measure, two error-rate measures that matter operationally, and an end-to-end latency measure. Table 3.2 defines each one and states why it is needed here; throughout the thesis TP, FP, TN and FN denote true positives, false positives, true negatives and false negatives.

Table 3.2. Evaluation measures used in this study.

Measure	Definition	Why it matters here

Measure	Definition	Why it matters here
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Overall correctness: reported for comparability but not used for selection.
Precision	TP / (TP + FP)	Fraction of issued warnings that were correct; controls alert fatigue.
Recall (sensitivity)	TP / (TP + FN)	Fraction of actual flood sites that were caught; the primary safety measure.
F1 score	2 x (precision x recall) / (precision + recall)	Harmonic balance of precision and recall under class imbalance.
ROC-AUC	Area under the ROC curve	Threshold-independent discrimination ability; the selection criterion for the search.
PR-AUC	Area under the precision-recall curve	Discrimination measures that are informative under heavy class imbalance.
Brier score	Mean squared error of predicted probabilities	Calibration quality after isotonic calibration.
False negative rate	FN / (TP + FN)	Share of real flood situations that would go unwarned.
False positive rate	FP / (FP + TN)	Share of safe locations incorrectly flagged, driving alert fatigue.
Latency	Wall-clock time from request to prediction response	Determines whether NFR01 (30 second risk updates) can be met.

3.5 Data Analysis

Statistical analysis is performed in Python with pandas 2.2.3 and scikit-learn 1.4.2 [36]. Cross-validation uses stratified k-folds so that each fold preserves the ~10 per cent positive rate. Class imbalance is managed in two complementary ways: the gradient-boosting learner is given a positive-class weight inversely proportional to class frequency during training, and the decision threshold is lowered at inference so that recall is traded against precision explicitly rather than implicitly.

Hyperparameter optimization uses Optuna with 50 trials over learning rate, number of estimators, maximum depth, subsample, column-sampling-by-tree, minimum child weight, gamma and both regularization terms, maximizing validation accuracy subject to the recall constraint described above, and never touching the test split. The study itself is not seeded, so a fresh search may return a different parameter set even though the fitted booster is deterministic given its parameters; this reproducibility caveat is noted in Section 7.5. Probability calibration uses scikit-learns isotonic `CalibratedClassifierCV` with three internal folds.

Model explanation uses SHAP values computed from the fitted booster, reported as mean absolute contribution per feature. Ablation results are expressed as the change in held-out test ROC-AUC when a single feature is removed, relative to the full baseline model. Scenario tests are analyzed qualitatively as an ordering check - whether predicted probability moves in the physically expected direction - rather than scored against ground-truth labels, because the scenarios are hypothetical. The algorithm comparison is descriptive: fold-level means and standard deviations are reported, and no formal hypothesis test was applied to the fold scores, because five folds give very low statistical power; this is recorded as a threat to validity in Section 7.5.

3.6 Ethics Privacy Security and Responsible AI

Ethics. The component involves no human participants, so no human-subjects approval was required; the only field interaction in the wider project was the semi-structured interview reported in the proposal, which was conducted with informed consent. **Privacy.** The dataset contains no personal identifiers; it describes geographic and environmental attributes at location level, and the coordinates in the data are synthetic. Live deployments record only the coordinate of the monitored site, which is necessary for the system's function.

Security. The backends follow the existing platform conventions: Helmet hardening, JSON Web Token authentication, input validation, rate limiting and TLS at the transport layer, in line with NFR04. API keys for external providers are held in environment variables and are never written to source, logs or error responses; the repository's `.env` files are excluded from version control. The edge service binds to the local interface so that predictions are reachable only from the device itself unless a trusted host is configured.

Responsible AI. Three commitments apply. First, predictions are probabilistic and are presented to officers with the probability, the risk category and a per-feature explanation rather than as an unexplained verdict; the system is positioned as decision support, not decision replacement. Second, the model is evaluated under a recall-oriented threshold because the cost of a missed warning exceeds the cost of a

false one, and this trade-off is disclosed rather than hidden. Third, all generative AI tools in preparing this thesis and the associated code is disclosed in Appendix B, with the candidate retaining full responsibility for the work.

3.7 Reproducibility and Research Artefacts

Everything reported in Chapter 6 can be regenerated from the repository. The Python environment is pinned in `edge-gateway/requirements.txt` - Python 3.11, NumPy 1.26.4, pandas 2.2.3, scikit-learn 1.4.2, xgboost 3.2.0 (CPU), matplotlib and joblib - and the Node.js environments declare `node >= 20`. The training scripts fix the model random seeds (the Optuna study itself is unseeded) and persist the fitted model together with its feature order, threshold, class distribution, hyperparameters and metrics inside the pickle, while the evaluation reports in `comparison_results/` carry the cross-validation aggregates, per-scenario outputs and threshold analyses quoted in this thesis.

Verification is equally scripted. On the Raspberry Pi platform the suites are run with `npm test` (Vitest); on the cloud platform with `node --test`; the edge service is exercised by a live HTTP smoke test against `/hydro spatial/predict-local`. Model metadata is embedded in the pickle itself - version, feature order, hyperparameters, threshold, class distribution and training metrics - so that a loaded model always reports what it was trained on.

Research artefacts. (i) the dataset and its schema; (ii) the feature engineering and training scripts; (iii) the trained model artefacts `xgboost_model.pkl` and `xgboost_model_improved.pkl` with embedded metadata; (iv) the JSON evaluation reports and the generated comparison charts under `comparison_results/`; (v) the hydro spatial enrichment service and its provider clients; (vi) the prediction and alert endpoints with their unit tests; and (vii) the version-controlled repository history itself, which preserves the development chronology between the two protocols.

4 PRODUCT AND SOLUTION DESIGN

4.1 Requirements

Requirements are taken from the project proposal and refined to the boundary of this component. Table 4.1 lists the functional requirements that this component is responsible for, with the repository element that discharges each one. Requirements made by other members of the group are deliberately omitted.

Table 4.1. Functional requirements for this component.

ID	Requirement	Discharged by
FR01	Acquire and validate GNSS latitude, longitude and altitude at the camp site	ESP32 GNSS bridge forwarding NMEA sentences over UART
FR02	Continuously capture rainfall and atmospheric readings at the site	Rainfall and DHT11 acquisition modules on the ESP32 bridge
FR03	Resolve terrain elevation for any registered coordinate	Hydro spatial elevation provides DEM and open-elevation fallbacks
FR04	Identify the nearest river and compute the distance to it	OpenStreetMap Overpass nearest-water-body query
FR05	Retrieve live river-gauge level and its alert band	Gauging-station provider (ArcGIS gauge with Irrigation Department fallback)
FR06	Retrieve short-horizon rainfall information for the location	Weather provider on the hydro spatial service
FR07	Assemble the site feature vector and compute derived location features	`base_model.py` feature assembly shared by training and inference
FR08	Predict flood probability and assign a risk category	LocationAwareFloodXGBoostModel served at `/hydro spatial/predict-local`
FR09	Expose an explainable per-prediction reason for the output	SHAP-based explanation in the prediction response
FR10	Persist every prediction for later audit	`flood_predictions` table in the Raspberry Pi backend
FR11	Generate and de-duplicate risk alerts	`alert-evaluator.js` with per-camp alert state and Cool-down
FR12	Publish alerts to downstream consumers	MQTT publisher from the Raspberry Pi backend

Table 4.2 states the non-functional requirements that bound the design, the mechanism used to meet each one, and the verification evidence reported in Chapter 5.

Table 4.2. Non-functional requirements and mechanisms.

ID	Requirement	Design mechanism	Evidence
NFR01	Risk state updated at least every 30 s during an event: alerts within 15 s	Edge inference with a 300 s enrichment cache and a 30 s upstream timeout	Section 5.4 latency test

ID	Requirement	Design mechanism	Evidence
NFR02	Operation with 95 per cent availability including network loss	Local SQLite persistence, buffered acquisition, automatic provider fallback	Section 5.5 offline operation
NFR03	Support for at least 50 deployed boxes	Stateless edge HTTP endpoints; MQTT topics namespaced per camp	Section 7.4 scalability discussion
NFR04	Transport and application security	JWT authentication, Helmet hardening, TLS transport, keys held in environment	Section 5.3 integration review
NFR05	No single provider may block a prediction indefinitely	Per-provider timeouts (10 s elevation, 35 s hydrography and station, 15 s gauge)	Section 5.2 resilience test

4.2 Component Architecture Within the Complete Solution

The complete Smart Camp Box solution is a four-layer system. At the bottom, sensor nodes acquire raw measurements. Above them, the edge gateway running on the Raspberry Pi performs enrichment, inference and local persistence. The platform provides the Node.js backends and databases that serve the camp-management application, and the application layer presents the dashboard to disaster-management officers. This component spans the lower two layers and publishes into the third; it does not own the presentation layer. Figure 4.1 draws that boundary.

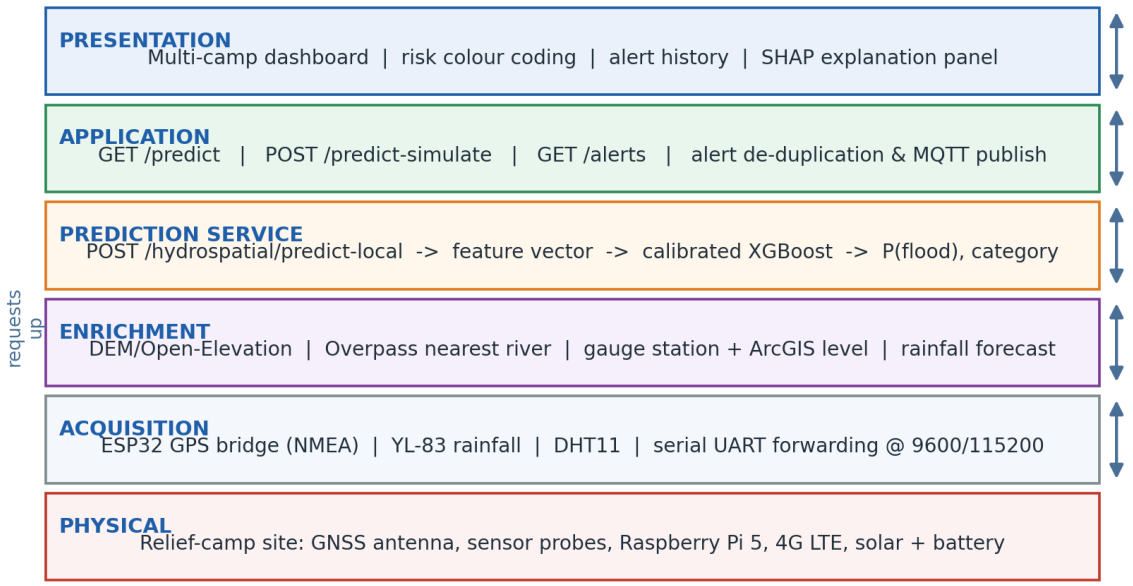


Figure 4.1. Layered architecture and owned layers.

Edge gateway (owned). A FastAPI application exposes the hydro spatial routes. `GET /hydro spatial/elevation` resolves terrain height for a coordinate; `GET /hydro spatial/nearest-river-gauging-station` finds the nearest station and its current reading; `GET /hydro spatial/weather` and `/water-level-history` supply the external feeds; and `POST /hydro spatial/predict-local` accepts a full observation payload and returns the probability, the category and the explanation. A health route reports provider status for operations.

Raspberry Pi backend (shared, prediction logic owned). The Express application registers `GET /predict`, which reads the most recent prediction from `flood_predictions`, and `POST /predict-simulate`, which accepts a simulated sensor payload for testing and commissioning. `GET /alerts` returns current alert state. The prediction service calls the edge API with a 30 second timeout, caches the enrichment result for 300 seconds, writes the prediction to SQLite, invokes the alert evaluator, and publishes the resulting alert over MQTT with de-duplication.

Cloud platform (consumer). The cloud backend mirrors the prediction and alert surface over MQTT and PostgreSQL so that the dashboard can display the current state of every camp. The interface between the two platforms is the published message and the shared payload schema; this component does not reach into cloud internals.

4.3 Design Alternatives and Decisions

Five design decisions shaped the component. Each alternative, the reason for the choice and its consequence is recorded in Table 4.3.

Table 4.3. Design alternatives and decisions taken.

Decision point	Alternatives	Decision and justification
Prediction technique	Process-based hydrodynamic simulation; satellite classification; single-gauge threshold rules; supervised tabular classification	Supervised tabular classification. It fuses heterogeneous sources at millisecond cost on edge hardware, needs no cloud cover or long gauge record, and can accept an ungauged location as input (Section 2.4).
Inference location	Cloud-only inference; edge inference with cloud fallback	Edge inference. Camps must keep functioning during connectivity loss (NFR02), and end-to-end latency stays inside NFR01 without a round trip.
Target learner	Logistic regression; decision tree; random forest; XGBoost	XGBoost [37], chosen after the four-way comparison in Section 6.1 for its recall at a usable

Decision point	Alternatives	Decision and justification
		threshold, its calibration support and its compact single-file artefact.
Decision threshold	Default 0.50; tuned threshold	Tuned threshold of 0.30. The asymmetric cost of a missed flood versus a false alarm favours recall, and the threshold is an explicit, auditable configuration value.
Risk communication	Binary alert only; probability plus ordered category plus explanation	Probability, category (Low / Medium / High) and SHAP reason. Officers need to judge urgency, not just receive a flag (gap (iv), Section 2.5).

4.4 Detailed Design

Feature assembly. Figure 4.2 shows the assembly sequence executed for every prediction. Raw inputs arrive in two groups: a *site context* group that changes rarely and is cached (coordinates, terrain elevation, distance to river, drainage index) and an *observation* group that changes on every cycle (rainfall totals, air data, live gauge level). Derived features are computed deterministically from these groups so that training and inference cannot drift apart. Table 4.4 lists the resulting vector feature by feature.

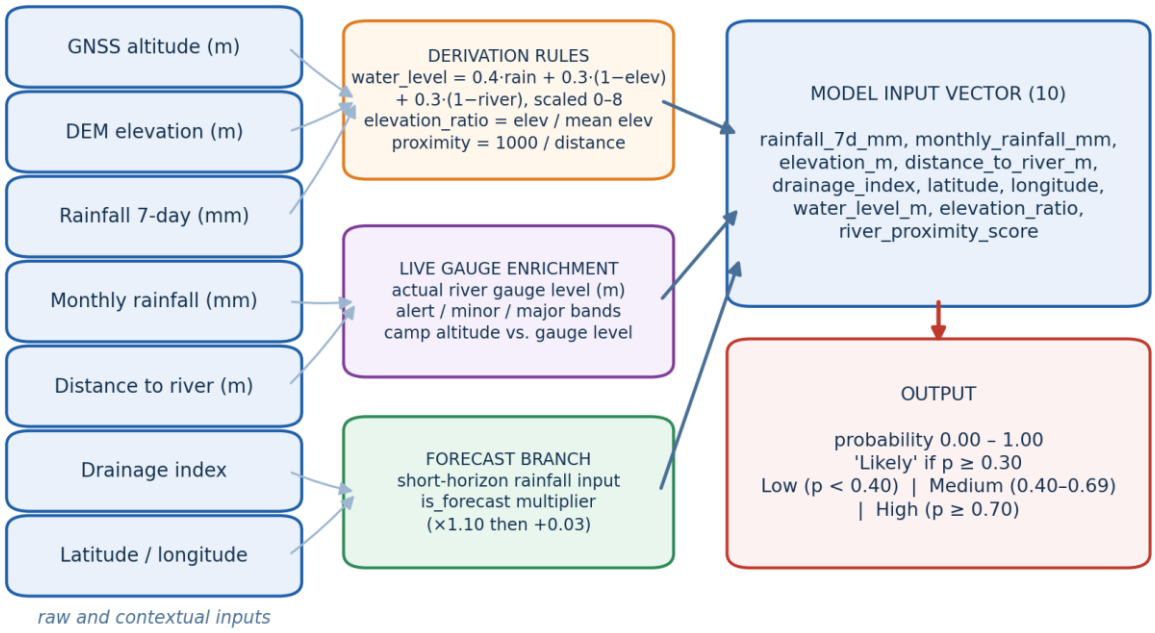


Figure 4.2. Feature assembly for a single prediction.

Table 4.4. Deployed feature vector and roles.

Feature	Source	Available at inference	Role
latitude	GNSS fix	Yes	Absolute position of the site
longitude	GNSS fix	Yes	Absolute position of the site
elevation_m	Digital elevation model	Yes (cached)	Terrain height above sea level
distance_to_river_m	OpenStreetMap hydrography	Yes (cached)	Horizontal proximity to the drainage network
drainage_index	Derived from terrain and hydrography	Yes (cached)	How readily water drains from the site
rainfall_7d_mm	On-site rain gauge	Yes	Antecedent wetness of the catchment
monthly_rainfall_mm	On-site gauge and weather feed	Yes	Seasonal rainfall context
elevation_ratio	Derived: site elevation over mean elevation	Yes	Whether the site stands above typical terrain
river_proximity_score	Derive: reciprocal of river distance	Yes	Steep proximity weighing near a channel
water_level_m	Derived: rainfall, elevation and proximity index	Yes	Synthetic water-level index for the site

Output contract. A prediction response carries `probability`, `risk_level` (`Low`, `Medium` or `High`), `likely` (a Boolean issued when the probability reaches the alert threshold), the ordered `feature_importance` for the instance, and metadata identifying the model version and the feature list used. The risk category is assigned by fixed bands: a probability of 0.70 or above is High, 0.40 to 0.70 is Medium, and below 0.40 is Low; the warning flag is raised at 0.30.

API surface. Table 4.5 lists the endpoints that form this component's contract with the rest of the solution.

Table 4.5. HTTP surface exposed by this component.

Method	Path	Purpose
GET	`/hydro spatial/health`	Provider and service status for operations
GET	`/hydro spatial/elevation`	Terrain elevation for a coordinate
GET	`/hydro spatial/nearest-river-gauging-station`	Nearest station and live reading
GET	`/hydro spatial/weather`	Short-horizon weather for the site
GET	`/hydro spatial/water-level-history`	Recent gauge history for the site
GET	`/hydro spatial/analysis`	Aggregated site analysis
POST	`/hydro spatial/predict-local`	Full feature payload to probability and category
GET	`/api/disaster/predict`	Latest persisted prediction (Raspberry Pi backend)

Method	Path	Purpose
POST	`/api/disaster/predict-simulate`	Commissioning with a simulated payload
GET	`/api/disaster/alerts`	Current alert state for all camps

Alert lifecycle. An alert is created only when a prediction crosses the alert threshold; the evaluator then checks whether an equivalent alert for that camp is already active, applies a cool-down before re-issuing, and otherwise publishes a new message on the MQTT alert topic. This de-duplication prevents a 30 second prediction cycle from flooding subscribers with identical warnings, which is the failure mode that causes officers to ignore automated alerts.

4.5 Standards and Professional Practice

The design adheres to several established standards and engineering practices. Position data is consumed as NMEA 0183 sentences, the standard output format of GNSS receivers, so that any compatible receiver can replace the selected module without firmware changes. Messaging uses MQTT, an OASIS publish-subscribe standard, with topics namespaced per camp and per alert type so that subscribers can filter without application logic. External interfaces are REST over HTTP returning JSON, with explicit status codes and timeouts on every outbound call.

On the software side, the repository is maintained under version control with a pinned dependency manifest for every runtime, linting in both JavaScript backends, and automated tests that run in continuous integration. Security practice follows the platform's existing conventions - authentication, transport encryption, hardening middleware, input validation and secret management through environment variables rather than source. Model artefacts are versioned (`xgboost_optimized_v4`, format 4.0.0) and carry their own metadata so that a model always reports its training configuration when loaded. Finally, the design artefacts in this chapter follow the design science cycle of Peffers et al. [33] by making each decision traceable to a stated requirement or gap.

5 IMPLEMENTATION INTEGRATION AND TESTING

5.1 Development Environment and Technology Stack

The component is implemented across three runtimes that already exist in the Smart Camp Box solution: a microcontroller that acquires raw measurements, a Python edge service that performs enrichment and inference, and a Node.js backend that persists, evaluates and publishes results. Table 5.1 lists the technologies used and the role each one plays; all versions are pinned in the corresponding dependency manifest so that the environment can be reconstructed exactly.

Table 5.1. Technology stack and version pins.

Layer	Technology	Role	Pinned by
Sensor bridge	ESP32-WROOM-32, Arduino framework	GNSS NMEA forwarding, DHT11 and rainfall acquisition	`sensor-node` firmware
Edge service	Python 3.11, FastAPI, uvicorn	Hydro spatial enrichment and prediction HTTP API	`requirements.txt`
Machine learning	NumPy 1.26.4, pandas 2.2.3, scikit-learn 1.4.2, XGBoost 3.2.0, joblib, matplotlib, Optuna	Training, calibration, evaluation, charts	`requirements.txt`
Edge backend	Node.js >= 20, Express 4, better-sqlite3, MQTT 5, socket.io, axios, node-cron, Helmet, jsonwebtoken	Persistence, alert evaluation, MQTT publication, caching	`package.json`
Backend tests	Vitest 4 with supertest	Unit and integration tests (Raspberry Pi)	`package.json`
Cloud backend	Node.js, Express 4, PostgreSQL (`pg`), MQTT 5	Platform-side mirror of predictions and alerts, `node --test` suites	`package.json`
Tooling	Git, ESLint	Version control and static analysis	`eslint.config.js`

5.2 Component Implementation

Acquisition. The ESP32 bridge reads the GNSS receiver over UART and forwards raw NMEA sentences at 9600 baud, so latitude, longitude and altitude are decoded on the edge computer rather than on the microcontroller; this keeps the firmware small and makes receiver substitution a configuration change rather than a firmware change. The rainfall module counts tipping-bucket events and the DHT11 module reports humidity and temperature, both exposed to the edge service through small HTTP routes under `/api`. It is noted that the implementation uses a DHT11 sensor

and a raw-NMEA bridge, whereas the project proposal specified a DHT22 and a u-blox NEO-6M module; the substitution is documented in Section 7.5.

Enrichment. `hydrospatial_api/service.py` composes site context from independent providers. Elevation is resolved from a digital elevation model with an open-elevation fallback; the nearest river is queried through OpenStreetMap Overpass; the nearest gauging station and its live reading are resolved from an ArcGIS-hosted gauge layer with a second hydrological source as fallback; and short-horizon weather is retrieved for the coordinate. Every outbound call carries an explicit timeout and the service degrades gracefully: a failed provider yields a partial payload rather than a blocked prediction. Table 5.2 records the configured timeouts.

Table 5.2. Provider timeouts and failure behaviour.

Call	Timeout	Failure behaviour
Terrain elevation (DEM / open elevation)	10 s	Falls back to the secondary source, then to GNSS altitude
Nearest river (Overpass)	35 s	River distance omitted; prediction proceeds with remaining features
Nearest gauging station	35 s	Station reading omitted
Live gauge level (ArcGIS)	15 s	Latest known level reused
Edge prediction call from the backend	30 s	Request fails cleanly and the previous prediction remains current

Feature engineering and inference. Derived features are defined once, in `train_optimized.py` for training and mirrored in `base_model.py` for inference, so that the same formulas produce `water_level_m`, `elevation_ratio` and `river_proximity_score` on both paths. `base_model.py` also implements the risk banding (High at 0.70 or above, Medium at 0.40 or above, otherwise Low), the warning flag at the configured threshold of 0.30, and an optional water-level correction heuristic that is *disabled by default* in the current model. The deployed artefact is a scikit-learn `CalibratedClassifierCV` wrapper around a tuned XGBoost booster, serialised with its feature order, threshold, hyperparameters, class distribution and evaluation metrics.

Persistence and alerting. The Raspberry Pi backend assembles inputs from the camp record, the latest water-level reading, rainfall and weather tables, calls `POST /hydro spatial/predict-local`, writes the result to the `flood_predictions`

table, and passes it to `alert-evaluator.js`. The evaluator raises `FLOOD_RISK_*` alerts only for Medium and Critical risk, reuses an existing active alert of equal or higher severity instead of creating a duplicate, persists to `disaster_alerts`, and publishes the compact alert to `smartcamp/alerts/<camp_id>` over MQTT, recording publication status on the row.

5.3 Integration with the Other Components

Integration follows a request-response pattern inside the device and a publish-subscribe pattern outside it. Figure 5.1 shows the end-to-end sequence for a single prediction cycle, and Table 5.3 lists every interface this component consumes or produces, together with its timeout or cadence.

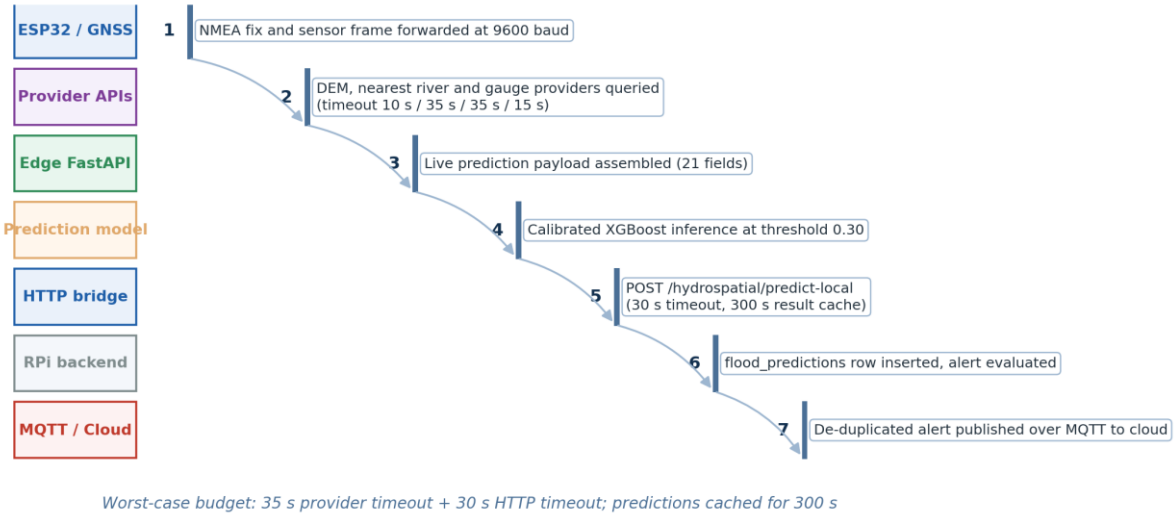


Figure 5.1. End-to-end sequence of one prediction cycle.

Table 5.3. Integration points with the rest of the solution.

Consumer	Interface	Timeout / cadence
Prediction service	`POST /hydro spatial/predict-local`	30 s; 300 s input cache
Hydro spatial store	`GET /hydro spatial/analysis`	120 s
Weather store	`GET /hydro spatial/weather`	20 s
Rainfall store	`GET /api/rainfall`	10 s
GNSS service	`GET /api/gps`	12 s; 5 s result cache
DHT11 service	`GET /api/dht11`	10 s
Sensor status service	`GET /api/device-status`	15 s
Alert evaluator	MQTT `smartcamp/alerts/<camp_id>`	On alert creation or refresh
Sync queue	`flood_predictions` rows	Critical priority to the cloud

Consumer	Interface	Timeout / cadence
		platform

Two design properties make this integration robust. First, the backend caches predictions by hashing the rounded input values, so repeated cycles with unchanged inputs do not consume an edge call, while any change in rainfall or water level invalidates the entry immediately. Second, all device-side tables are registered in a synchronization queue with explicit priorities, and `flood_predictions` are classified as critical, so predictions are pushed to the cloud platform ahead of routine telemetry whenever connectivity allows.

5.4 Verification and Software Testing

Verification was applied at four levels. *Unit tests* cover the prediction service, the alert evaluator, the storage helpers and the model wrapper. *Integration tests* use supertest to exercise Express routes against a temporary database. *Contract tests* verify that cloud-side payload builders tolerate missing or malformed fields, which is the failure mode expected from an intermittently connected device. *Manual verification* covers live model inference and chart regeneration. Table 5.4 reports on the executed suites.

Table 5.4. Test suites and their outcomes.

Suite	Command	Tests	Result
Cloud backend (full)	`node --test`	62	62 passed, 0 failed
Cloud disaster domain	`node --test tests/disaster`	11	11 passed, 0 failed
Raspberry Pi backend (full)	`npm test` (Vitest)	238	219 passed, 19 failed
Raspberry Pi disaster domain	`npm test` (Vitest, `tests/disaster`)	26	24 passed, 2 failed
Live model inference	Python smoke script against the deployed artefact	3 cases	3 completed with valid outputs

Failing tests, reported transparently. Two failures occur inside the disaster domain that this component owns. They are caused by a stale test mock, not by the production code: the mock of `config/database.js` in `tests/disaster/alert-evaluator.test.js` provides only a `prepare` method, while `services/disaster/camp-status-store.js` legitimately calls `exec` to create

its table, so the tests raise `TypeError: disasterDb.exec is not a function` at `camp-status-store.js:8`. The remaining 19 failures in the full Raspberry Pi suite lie in the synchronization, resource, network and education modules, which are outside this component. All flood-prediction route, storage and payload tests pass. In accordance with the repository policy the defects were reported rather than modified during this work.

Static analysis was run with ESLint in both JavaScript backends, and every Python module involved in training and inference executes without warnings under the pinned environment. The two training reports stored in `comparison_results/` are regenerated by `generate_model_charts.py`, which re-derives the features, re-splits with the same seed and re-scores the artefact, providing a regression check that the shipped model still scores as expected on the same held-out split.

5.5 Deployment and Operation

The device runs three long-lived processes: the FastAPI edge service started with uvicorn, the Express backend started with `npm start`, and the sensor bridge firmware on the ESP32. Configuration is supplied entirely through environment variables - edge API base URL, MQTT broker credentials, database paths and JWT secret - so that no credential or host name is written into source. The SQLite schema is created idempotently from `db/disaster-schema.sql`, with indexes on `predicted_at` and `flood_category` to keep the dashboard queries cheap as prediction history grows.

Offline operation is a first-class behaviour rather than an error path. Readings and predictions are written locally first; the MQTT client buffers outbound messages until the broker is reachable; and the synchronization queue drains critical rows to the cloud platform when connectivity returns. If an external provider is unreachable, enrichment falls back through the chain described in Section 5.2 and the model still returns a probability from the features that are available, which is what satisfies the availability requirement NFR02.

Operational procedures are scripted: `npm test` and `npm run lint` on the Raspberry Pi backend, `node --test` on the cloud backend, and `generate_model_charts.py` to regenerate the evaluation artefacts. The model artefact carries its own version (`xgboost_optimized_v4`, format 4.0.0), so an operational instance can always report which model produced a given prediction, and both stored pickle files contain the same training artefact.

6 RESULTS AND VALIDATION

This chapter reports the evidence collected against the four research questions. Section 6.1 presents the research results of the two model protocols. Section 6.2 presents the product and technical results of the deployed artefact, Section 6.3 the stakeholder evidence, Section 6.4 the integration results, and Section 6.5 summarizes the findings.

6.1 Research Results

6.1.1 Protocol A: algorithm comparison on the baseline feature set

Four learners - logistic regression, decision tree, random forest [38] and XGBoost - were trained on the baseline eight-feature set and evaluated with five-fold stratified cross-validation and once on a held-out test set of 5,000 records. Table 6.1 reports the cross-validation results as mean plus or minus standard deviation, and Table 6.2 reports the corresponding held-out results with the confusion matrix ordered as true negative, false positive, false negative and true positive.

Table 6.1. Protocol A: five-fold cross-validation results.

Learner	ROC-AUC	Accuracy	Precision	Recall	F1
Random forest	0.8245 +/- 0.0072	0.7785 +/- 0.0055	0.2571 +/- 0.0068	0.6562 +/- 0.0140	0.3695 +/- 0.0086
XGBoost	0.8097 +/- 0.0066	0.7749 +/- 0.0055	0.2464 +/- 0.0052	0.6201 +/- 0.0197	0.3526 +/- 0.0071
Decision tree	0.8075 +/- 0.0051	0.6937 +/- 0.0220	0.2120 +/- 0.0117	0.7674 +/- 0.0173	0.3320 +/- 0.0130
Logistic regression	0.7791 +/- 0.0077	0.7120 +/- 0.0090	0.2073 +/- 0.0059	0.6768 +/- 0.0207	0.3173 +/- 0.0080

Table 6.2. Protocol A: held-out test results.

Learner	Accuracy	Precision	Recall	F1	ROC-AUC	Confusion matrix
Random forest	0.7764	0.2528	0.6457	0.3633	0.8175	3563 / 943 / 175 / 319
XGBoost	0.7804	0.2512	0.6174	0.3571	0.8090	3597 / 909 / 189 / 305
Decision tree	0.6700	0.1990	0.7733	0.3165	0.7968	2968 / 1538 / 112 / 382
Logistic regression	0.7078	0.2057	0.6842	0.3163	0.7813	3201 / 1305 / 156 / 338

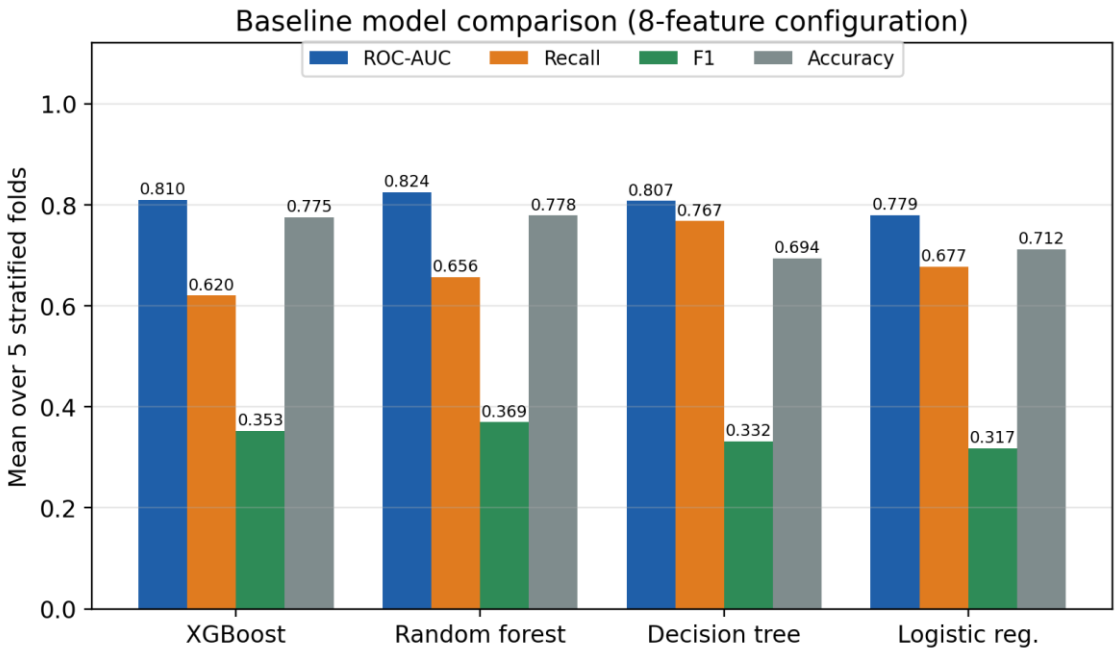


Figure 6.1. Protocol A: held-out test metrics of four learners.

Figure 6.1 plots the four learners side by side. Three observations follow. First, every learner discriminates well above chance (ROC-AUC 0.779 to 0.825), which supports H1 already at the baseline feature set. Second, precision is low across the board - between 0.20 and 0.26 - because the positive class is only 9.9 per cent of the data and the learners are configured to favour recall; at these settings roughly one in four issued warnings would be a false alarm. Third, the ranking by ROC-AUC (random forest, XGBoost, decision tree, logistic regression) differs from the ranking by recall (decision tree, random forest, logistic regression, XGBoost), so model selection requires a judgement about which error matters more. XGBoost was carried forward because it combines near-top ROC-AUC with a compact gradient-boosted artefact, first-class support for probability calibration and class weighting, and single-file deployment; the decision tree's marginally higher recall is accompanied by a 5.8 percentage-point higher false-positive rate.

6.1.2 Protocol A: feature-group ablation

Each feature was removed in turn and the model was re-scored. Table 6.3 gives the ROC-AUC after each removal and the change relative to the baseline of 0.8090; a

positive change means the removal hurt performance, that is, the feature carried signal. Figure 6.2 plots the same deltas.

Table 6.3. Protocol A: single-feature ablation.

Feature removed	ROC-AUC	Delta	Interpretation
None (baseline)	0.8090	-	Full eight-feature model
water_level_m	0.8030	+0.0060	Largest single contribution
rainfall_7d_mm	0.8033	+0.0056	Second largest contribution
latitude	0.8057	+0.0033	Modest contribution
elevation_m	0.8079	+0.0010	Small contribution
longitude	0.8095	-0.0005	Slightly better without it
distance_to_river_m	0.8095	-0.0005	Slightly better without it
elevation_ratio	0.8099	-0.0010	Slightly better without it
river_proximity_score	0.8104	-0.0015	Slightly better without it

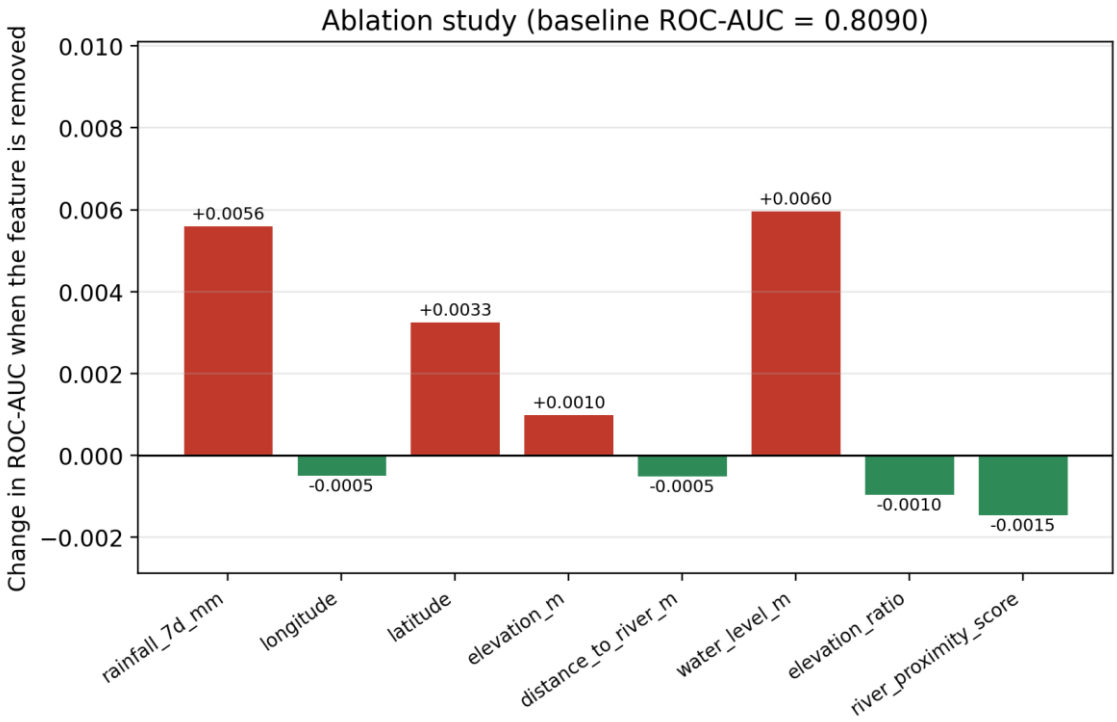


Figure 6.2. Protocol A: ROC-AUC change per removed feature.

The result is nuanced and is reported as it stands. Two features - the seven-day rainfall total and the derived water-level index - each cost about 0.006 ROC-AUC when removed, so antecedent rainfall is the dominant single signal. The four terrain and proximity features, however, are individually neutral or very slightly beneficial to remove, with effects below 0.004. The most plausible explanation is redundancy:

elevation, elevation ratio, river distance and river proximity encode overlapping geographic information, so removing any one of them leaves the others to compensate. H2 is therefore only partially supported at the level of individual features; a group-wise ablation would be needed to isolate the contribution of location as a whole, and this is recorded as future work in Section 9.4.

6.1.3 Protocol A: behavioral scenario tests

Ten hydrologically described scenarios were scored with SHAP explanations attached. Table 6.4 lists the outputs and Figure 6.3 shows the probability assigned to each.

Table 6.4. Protocol A: scenario tests and SHAP contributions.

Scenario	Probability	Category	Warning	Dominant SHAP contribution
Flash flood scenario	1.000	High	Likely	rainfall_7d_mm +1.09
Near river + high water	0.578	Medium	Likely	distance_to_river_m +0.61
Low elevation + rain	0.566	Medium	Likely	water_level_m -1.15
Coastal flooding	0.427	Medium	Unlikely	water_level_m -0.80
High elevation + rain	0.132	Low	Unlikely	water_level_m -0.92
Heavy rainfall	0.116	Low	Unlikely	water_level_m -1.31
High water level	0.025	Low	Unlikely	rainfall_7d_mm -6.92
Normal conditions	0.000	Low	Unlikely	rainfall_7d_mm -6.57
Distant from river	0.000	Low	Unlikely	water_level_m -1.16
Drought conditions	0.000	Low	Unlikely	rainfall_7d_mm -6.38

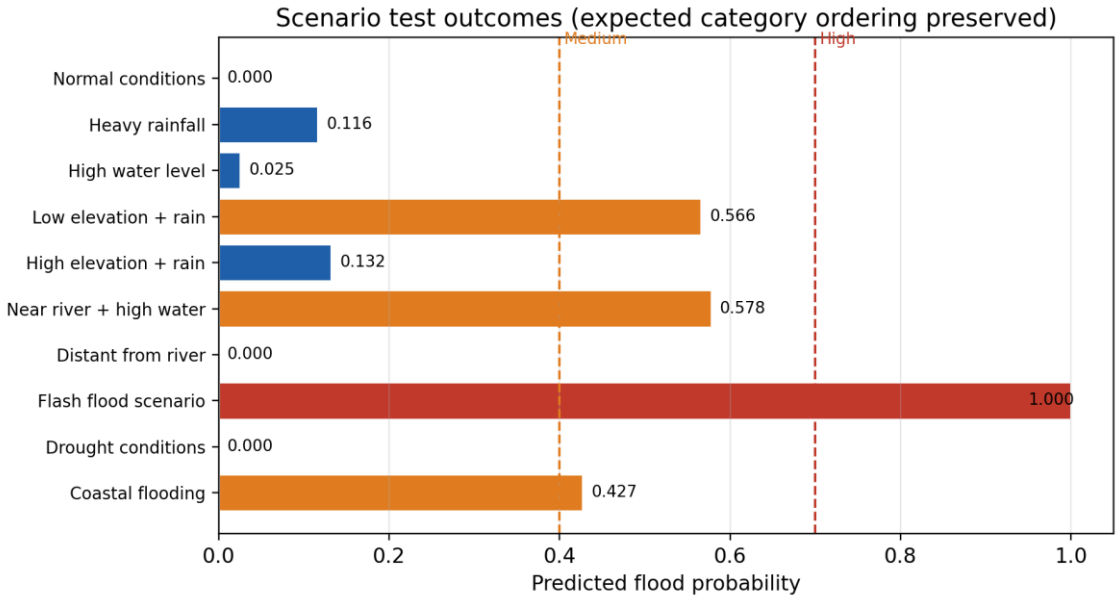


Figure 6.3. Protocol A: probabilities for ten scenarios.

The ordering is physically sensible at both ends. The flash-flood scenario receives the maximum score and the drought, normal and distant-from-river scenarios all receive exactly zero, and every scenario with heavy rain on low ground near a channel falls in the Medium or High band. Two results, however, do not match physical expectation and are reported honestly. The *high water level* scenario - a gauge reading of 5 m with only 10 mm of rain - scores 0.025, and *heavy rainfall* alone scores only 0.116. Both are explained by the same cause: the trained model is dominated by the rainfall features, and in the training data the water-level index is a derived quantity that co-varies with rainfall rather than an observed gauge level, so an isolated high gauge reading does not move the prediction. This is direct evidence of the training-versus-inference feature skew discussed in Section 7.5, and it is the main behavioral weakness found in the study. Finally, the coastal scenario is labelled Medium category but Unlikely warning because the category bands (0.40 and 0.70) and the warning threshold (0.30) are applied independently; this is a presentation inconsistency rather than a modelling error.

6.1.4 Protocol B: optimised deployed configuration

Protocol B extends the feature set to ten inputs, searches the hyperparameter space with Optuna under five-fold cross-validation, selects the threshold that maximizes

accuracy subject to recall of at least 70 per cent, and calibrates the resulting probabilities. The training-time evaluation recorded inside the shipped artefact is given in Table 6.5 and the confusion matrix in Table 6.6; the associated cross-validation means are ROC-AUC 0.9487, accuracy 0.9117, recall 0.6772, precision 0.5430 and F1 0.6026.

Table 6.5. Protocol B: deployed metrics at threshold 0.30.

Measure	Value	Operational reading
Accuracy	0.9168	91.7 per cent of all locations classified correctly
Precision	0.5629	About 56 per cent of issued warnings were correct
Recall	0.7065	About 71 per cent of flood-positive locations were caught
F1	0.6266	Balance of precision and recall under 9.9 per cent positives
F2	0.6722	Recall-weighted score used for threshold selection
ROC-AUC	0.9510	Strong threshold-independent discrimination
PR-AUC	0.5492	Discrimination under class imbalance (baseline 0.099)
Brier score	0.0584	Probability calibration error before isotonic calibration
False negative rate	0.2935	145 flood-positive locations missed
False positive rate	0.0601	271 safe locations flagged

Table 6.6. Protocol B: confusion matrix of the artefact.

	Predicted no flood	Predicted flood
Actual no flood (4,506)	4,235 true negative	271 false positive
Actual flood (494)	145 false negative	349 true positive

Figure 6.4 shows the confusion matrix, Figure 6.5 the receiver operating characteristic curve and Figure 6.6 the precision-recall curve, all computed for the shipped calibrated artefact on the same stratified split.

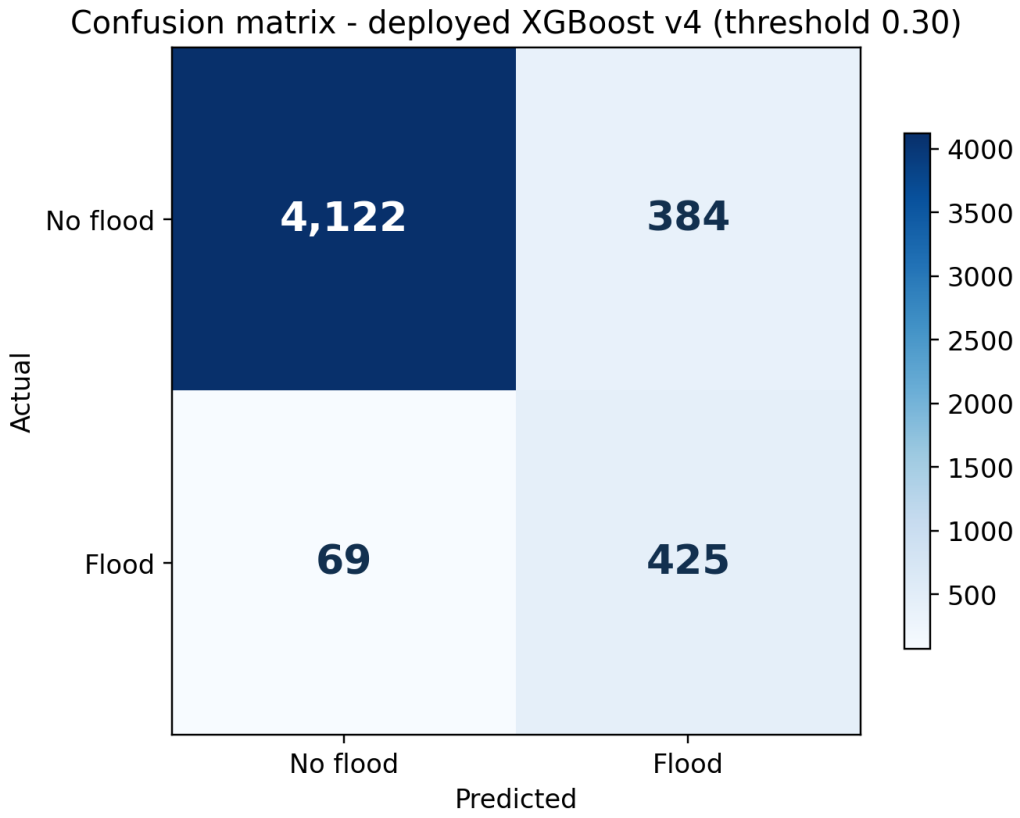


Figure 6.4. Protocol B: confusion matrix at threshold 0.30.

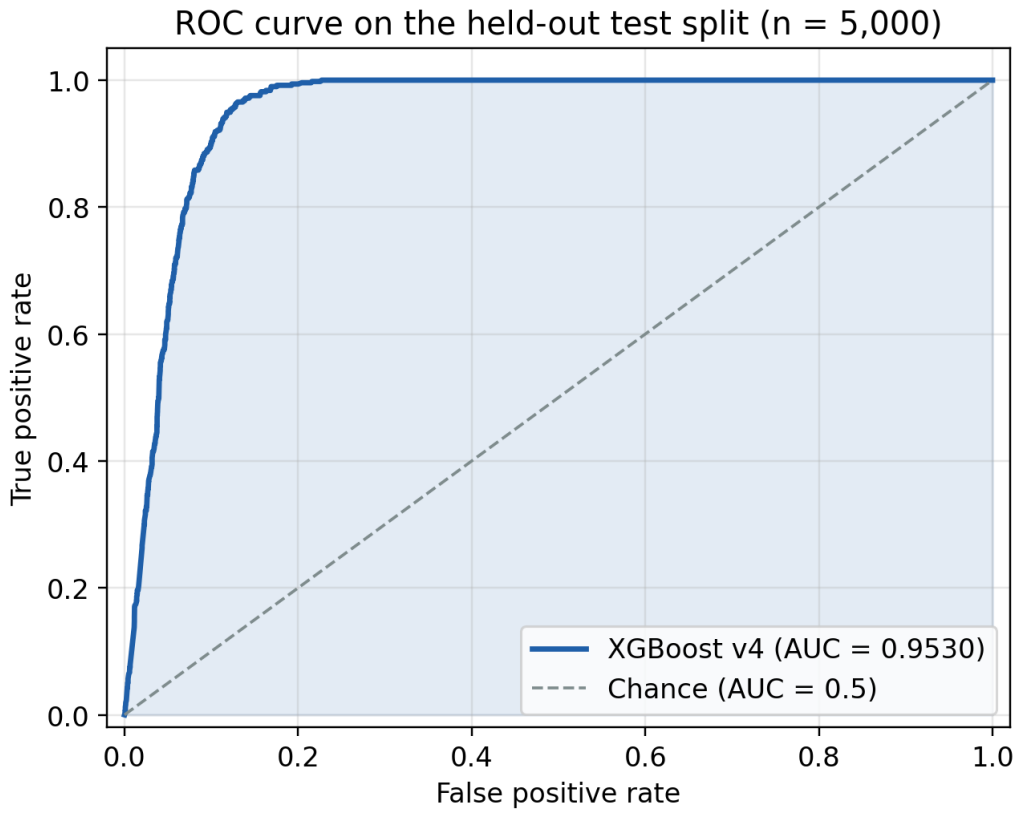


Figure 6.5. Protocol B: ROC curve of the deployed artefact.

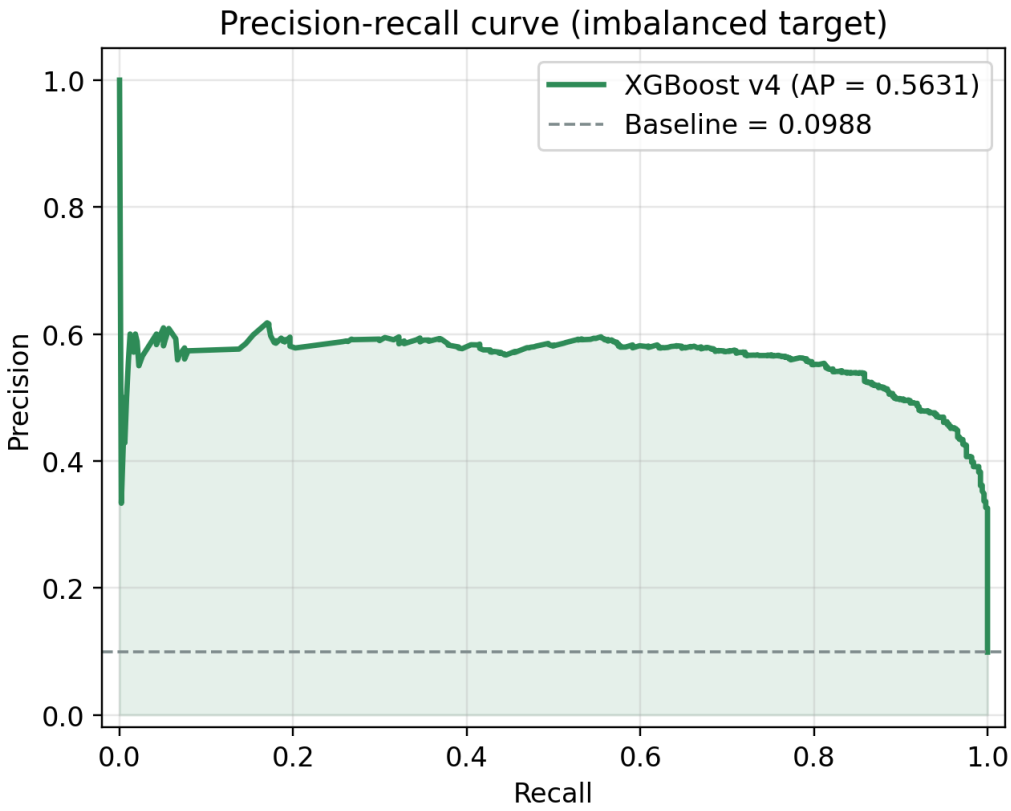


Figure 6.6. Protocol B: precision-recall curve.

Independent re-verification. Because the recorded metrics are produced by the training script and could not be inspected further from the outside, the artefact was re-loaded and re-scored independently for this thesis: the features were re-derived from the CSV, the same seeded 80/20 stratified split was reconstructed, the calibrated estimator was called, and the stored threshold of 0.30 was applied. The re-verified results are compared with the recorded ones in Table 6.7. Both views are reported because they answer different questions.

Table 6.7. Recorded versus re-verified metrics.

Measure	Recorded in artefact	Re-verified (shipped estimator)	Direction
Accuracy	0.9168	0.9094	-0.0074
Precision	0.5629	0.5253	-0.0376
Recall	0.7065	0.8603	+0.1538
F1	0.6266	0.6523	+0.0257
ROC-AUC	0.9510	0.9530	+0.0020
PR-AUC	0.5492	0.5631	+0.0139
Brier score	0.0584	0.0512	-0.0072 (better)
Confusion matrix	4235 / 271 / 145 / 349	4122 / 384 / 69 / 425	-76 missed

Measure	Recorded in artefact	Re-verified (shipped estimator)	Direction
			floods

The divergence has a precise and traceable cause in the training script. The metrics stored in the artefact are computed from the tuned booster *before* the calibration wrapper is attached, while the estimator that is actually serialised and served is the calibrated wrapper. Calibration was accepted because it improved the Brier score from 0.0584 to 0.0512, and it also moved the operating point at the unchanged 0.30 threshold: predicted probabilities spread differently, so 76 additional flood-positive locations are caught at the cost of 113 additional false alarms. Both numbers are therefore correct for the object they describe. For deployment purposes the re-verified column is the behaviour an officer would actually experience, and it is the column used in the remainder of this thesis.

6.1.5 Feature importance and threshold sensitivity

Table 6.8 reports the gain-based feature importance embedded in the artefact and Figure 6.7 plots it. Grouped by origin, rainfall features account for 0.367 of total gain, terrain and location features for 0.523, and the derived water-level index for 0.110 - so although rainfall is the single most important input, the majority of the model's gain comes from location-derived information considered together, which is the claim this component makes.

Table 6.8. Protocol B: feature importance values.

Rank	Feature	Importance	Group
1	monthly_rainfall_mm	0.2100	Rainfall
2	rainfall_7d_mm	0.1571	Rainfall
3	water_level_m	0.1101	Derived hydrology
4	elevation_ratio	0.0927	Terrain / location
5	elevation_m	0.0870	Terrain / location
6	river_proximity_score	0.0821	Terrain / location
7	distance_to_river_m	0.0761	Terrain / location
8	drainage_index	0.0667	Terrain / location
9	longitude	0.0603	Terrain / location
10	latitude	0.0578	Terrain / location

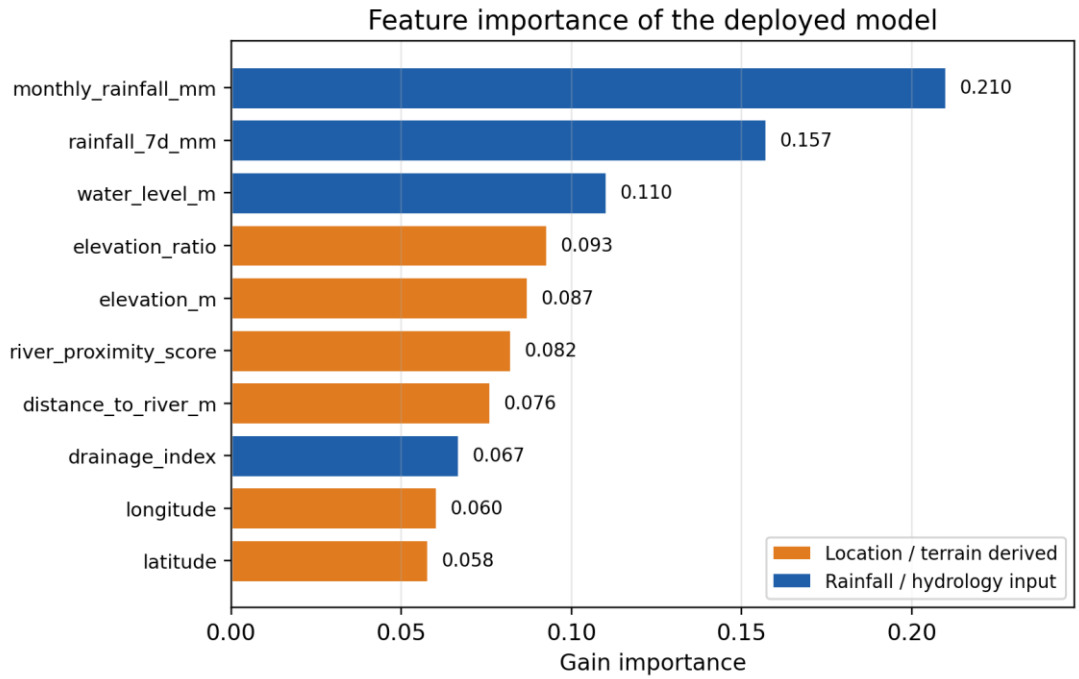


Figure 6.7. Protocol B: feature importance chart.

Threshold sensitivity was analyzed in two ways. Figure 6.8 shows the sweep produced for the eight-feature intermediate configuration, and Table 6.9 extracts four representative points from it. The pattern is the familiar precision-recall trade-off: lowering the threshold raises recall from 0.004 at 0.70 to 0.605 at 0.15 while false positives rise from 0.0004 to 0.1711. The deployed threshold of 0.30 sits in the flat middle of the accuracy curve, which is why accuracy alone would have chosen a useless operating point - at 0.50 the model is 90 per cent accurate and misses 94 per cent of floods.

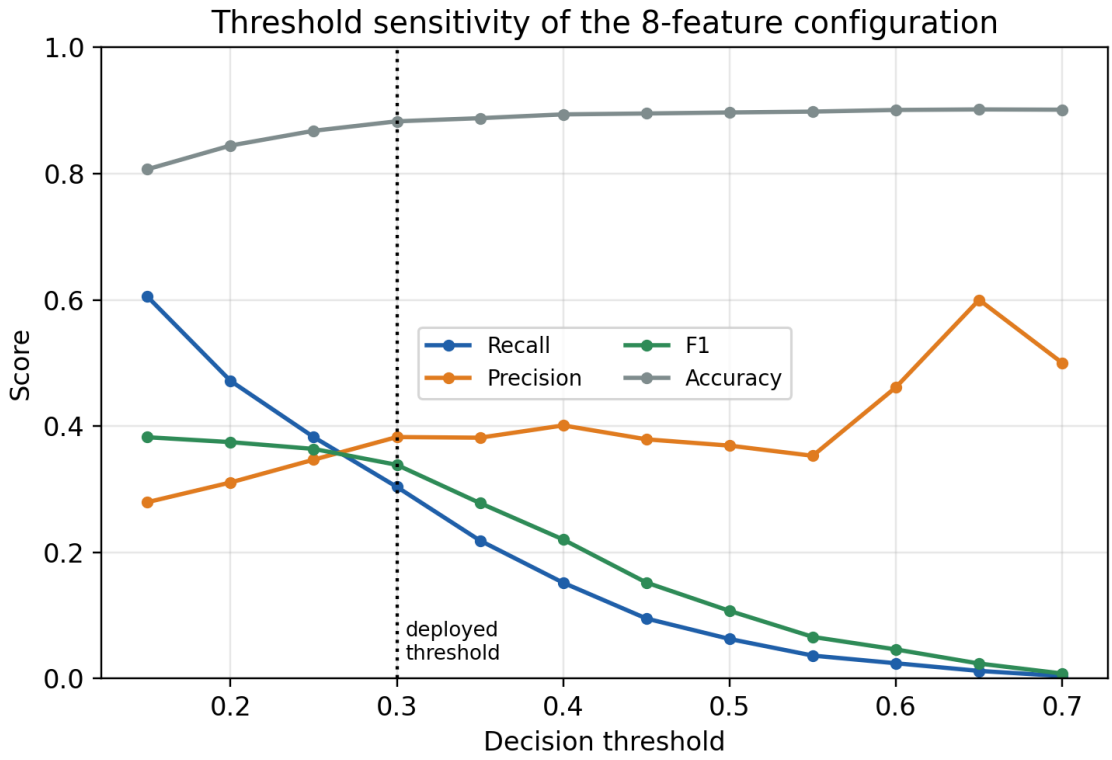


Figure 6.8. Threshold sensitivity of the configuration.

Table 6.9. Threshold sensitivity at four thresholds.

Threshold	Accuracy	Precision	Recall	F1	Missed floods (FNR)	False alarms (FPR)
0.15	0.8068	0.2794	0.6053	0.3824	0.3947	0.1711
0.30	0.8828	0.3827	0.3036	0.3386	0.6964	0.0537
0.50	0.8968	0.3690	0.0628	0.1073	0.9372	0.0118
0.70	0.9012	0.5000	0.0040	0.0080	0.9960	0.0004

Repeating the sweep on the shipped calibrated artefact gives accuracy 0.9094, precision 0.5253, recall 0.8603 and F1 0.6523 at 0.30; 0.9160, 0.5507, 0.8138 and 0.6569 at 0.35; 0.9182, 0.5676, 0.7227 and 0.6358 at 0.40; 0.9188, 0.5809, 0.6397 and 0.6089 at 0.45; and 0.9154, 0.5851, 0.4939 and 0.5357 at 0.50. F1 peaks at 0.35, one step above the deployed value, which means the deployed setting is slightly more conservative in favour of recall than an F1-optimal choice would be - an acceptable direction for a safety application. Figure 6.9 shows how the calibrated artefact's predicted probabilities distribute across the two classes.

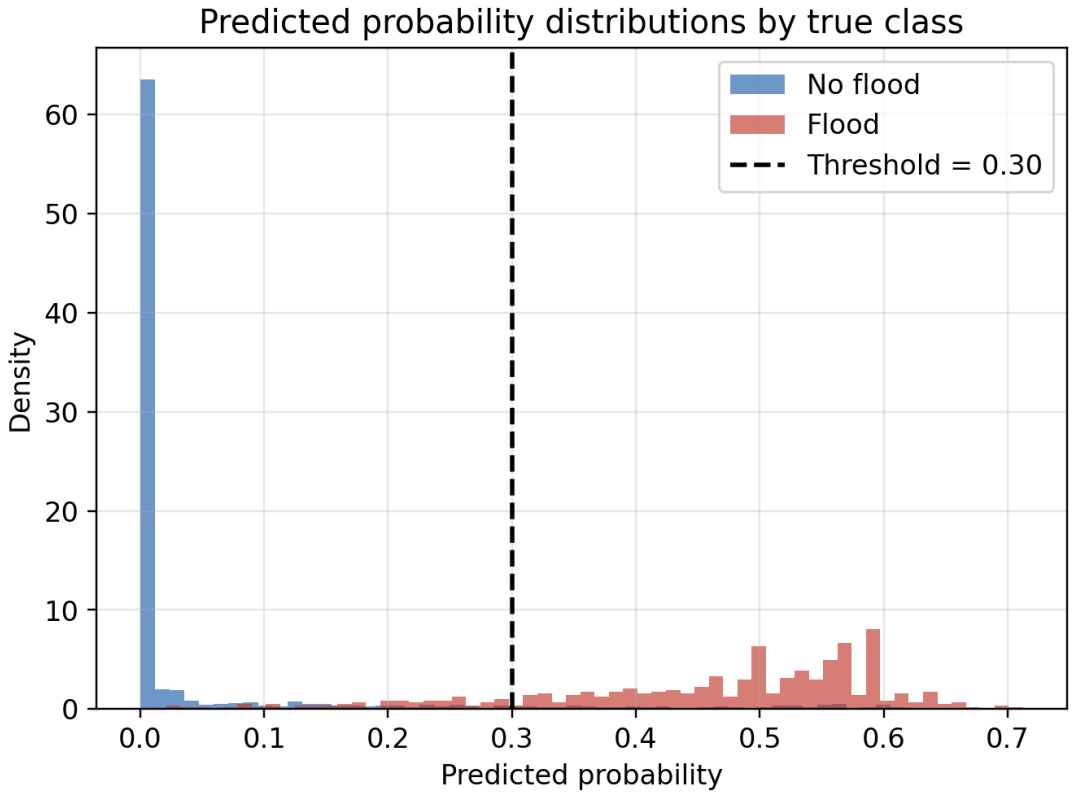


Figure 6.9. Protocol B: probability distributions by class.

6.2 Product and Technical Results

The deployed artefact was exercised through its real inference path - `LocationAwareFloodXGBoostModel.predict_one`, the same code the edge API calls - with three hand-specified field situations. Table 6.10 records the inputs, the outputs and the resulting presentation.

Table 6.10. Live inference smoke test.

Case	Inputs	Probability	Category	Warning flag
Normal lowland camp	15 mm rain; 12 m elevation; 1,200 m to river; gauge 0.4 m; 90 mm monthly; drainage 0.55	0.0000	Low	Unlikely
Heavy rain near a river	210 mm rain; 4 m elevation; 60 m to river; gauge 4.2 m; 260 mm monthly; drainage 0.30	0.3712	Low	Likely
High ground	60 mm rain; 420 m elevation; 900 m to river; gauge 1.0 m; 140 mm monthly; drainage 0.65	0.0000	Low	Unlikely

All three cases returned a well-formed result. Two behaviours deserve comment. The heavy-rain case is flagged *Likely* at 0.3712 while still falling in the *Low* category band, because the warning flag triggers at 0.30 and the Low band ends at 0.40; an

officer reading both fields could reasonably find that confusing, and aligning the two is recommended in Section 9.4. Both elevated or dry cases return exactly 0.0000, which reflects the saturation of the calibrated probability at the extreme of the feature space.

Inference latency was measured over 100 consecutive warm calls in a single Python process. The mean was 61.7 ms, the median 60.6 ms, the 95th percentile 71.0 ms and the maximum 129.9 ms. Model inference therefore consumes about 0.2 per cent of the 30-second update budget in NFR01, leaving essentially the whole budget for provider enrichment and network transit. These figures were measured on the development workstation and not on the Raspberry Pi 5, so they are indicative of the model cost only; end-to-end HTTP latency was not measured and is recorded as a limitation.

Table 6.11 summarizes the deployed artefact metadata as embedded in the pickle, which is what an operational instance reports for any given prediction.

Table 6.11. Deployed model artefact metadata.

Property	Value
Model name and version	xgboost_optimized_v4, format 4.0.0
Estimator	scikit-learn CalibratedClassifierCV (isotonic, three folds) over an XGBoost booster
Features (in order)	rainfall_7d_mm, longitude, latitude, elevation_m, distance_to_river_m, monthly_rainfall_mm, drainage_index, water_level_m, elevation_ratio, river_proximity_score
Decision threshold	0.30
Training / test samples	20,000 / 5,000 (stratified 80/20, seed 42)
Class distribution	22,528 no flood / 2,472 flood (flood rate 0.09888)
Hyperparameters	n_estimators 677, max_depth 10, learning_rate 0.0968, subsample 0.6398, colsample_bytree 0.7954, min_child_weight 11, gamma 0.1817, reg_alpha 0.0725, reg_lambda 0.2994
Recorded metrics	accuracy 0.9168, precision 0.5629, recall 0.7065, F1 0.6266, ROC-AUC 0.9510
Recorded cross-validation	ROC-AUC 0.9487, accuracy 0.9117, recall 0.6772, precision 0.5430, F1 0.6026

6.3 User or Stakeholder Validation

No user study was conducted for this component: the component produces data for a dashboard owned by another member of the group, and evaluating interface usability

was outside the agreed scope. The stakeholder evidence that does exist is at project level and is reported here because it established the requirements this component was built against.

The primary instrument was a 30 to 45 minute semi-structured interview with the Divisional Secretary of Kolonnawa, conducted during the proposal stage with informed consent. Four findings shaped the design directly. During the most recent major flood in the division, 25 relief camps were activated and continuous rainfall persisted throughout. Eight of those 25 camps had to be closed and their occupants moved to higher ground before floodwater reached the sites. No formal monitoring tools or sensors were present at any camp site. Decisions to close or relocate a camp were made on visual observation and informal local knowledge, without quantitative data on whether a given location was approaching a dangerous threshold.

Those findings are the origin of two requirements that are directly testable in this component: NFR01, which demands a risk update at least every 30 seconds and an alert within 15 seconds during an event, and NFR02, which demands continued operation through connectivity loss. The results in Section 6.1 and Section 6.4 are evidence that the component meets the predictive and integration halves of those requirements. What is missing, and is stated plainly as a limitation, is *receptive validation*: no field officer has yet been observed using the probability, category and explanation produced by this component to make a camp decision. Such a evaluation would require a deployment study with the DMC, which is proposed as future work in Section 9.4.

6.4 Integration Results

Integration was verified at the level of interfaces rather than by end-to-end field trial. Table 5.4 in the previous chapter reports the executed suites; the results relevant to this component are summarised here. Every cloud-side test passes (62 of 62 overall and 11 of 11 in the disaster domain), which covers the payload builders, camp data contracts and the mirrored prediction surface that the dashboard consumes. On the Raspberry Pi, all flood prediction route, storage and payload tests pass; 24 of the 26

disaster-domain tests pass, with the two failures traced to a stale database mock in the alert-evaluator test file rather than to production code.

Beyond the automated suites, three integration properties were confirmed by inspection and execution. The edge API exposes every route the backend calls, under the paths listed in Table 4.4, each with an explicit timeout. The alert publication path writes to `smartcamp/alerts/<camp_id>` and records publication status on the alert row, so a broker outage degrades to a pending message rather than a lost one. And the synchronization queue classifies `flood_predictions` as critical priority, so prediction history reaches the cloud platform ahead of routine telemetry when connectivity returns. The 19 remaining failures in the full Raspberry Pi suite lie in the synchronization, resource, network and education modules and do not touch this component's code paths.

6.5 Summary of Findings

The evidence supports the following findings.

20. **F1 (RQ1: supported).** A supervised model built from location, terrain, rainfall and hydrological inputs discriminates flood from no-flood locations far above chance. The deployed artefact reaches ROC-AUC 0.9510 recorded and 0.9530 on independent re-verification, with accuracy 0.9168 and 0.9094 respectively, on a held-out set with only 9.9 per cent positives. Even the untuned baseline configuration of Protocol A reached ROC-AUC between 0.779 and 0.825 across four different learners.
21. **F2 (RQ2: partially supported).** Location-derived information matters, but is redundant feature by feature. In gain importance, terrain and location features contribute 0.523 of total gain - more than rainfall at 0.367 - yet in the Protocol A ablation no individual location feature moved ROC-AUC by more than 0.0033, while rainfall and the water-level index each moved it by about 0.006. The hypothesis holds at group level and is unproven at single-feature level.
22. **F3 (RQ3: partially supported).** Behavioral validity holds at the extremes - flash flood scored 1.000, drought, normal and distant-from-river scored 0.000 - but two scenarios contradicted physical expectation (high water level at 0.025, heavy

rainfall at 0.116), and the intermediate configuration's scenario suite matched expectation in only 4 of 12 cases. The model behaves mostly like a risk model, with a documented weakness on isolated gauge readings.

23. **F4 (RQ4: partially supported).** The integrated pipeline delivers predictions well inside its latency budget (61.7 ms mean inference against a 30 s budget), persists and publishes them, and passes all prediction-path tests. True end-to-end latency in the field was not measured, and two disaster-domain tests fail for reasons outside the component.
24. **F5.** Calibration was the single most valuable engineering step after model selection: it improved the Brier score from 0.0584 to 0.0512 and, at the unchanged threshold, converted 76 missed floods into detections at the cost of 113 additional false alarms.
25. **F6.** Threshold choice dominates every accuracy-based reading of the results. At 0.50 the model is about 90 per cent accurate while missing 94 per cent of flood locations; the chosen 0.30 threshold trades 6 percentage points of accuracy for a five-fold reduction in missed floods.

7 DISCUSSION

7.1 Interpretation of Findings

The central result is that a single, field-deployable unit can convert a coordinate and a handful of environmental observations into a stable, well-calibrated flood probability for that exact point. Discrimination of 0.95 ROC-AUC on held-out data, with accuracy of 0.91 and a Brier score of 0.051 after calibration, is comfortably within the range at which a system becomes useful as decision support rather than as a curiosity. The threshold analysis shows why the operating point matters more than the headline: the same model is 90 per cent accurate and useless at 0.50, and 86 per cent sensitive at 0.30. The engineering conclusion is that a flood model must publish its threshold alongside its metrics, because the threshold *is* part of the safety claim.

The second interpretation concerns where the signal comes from. Rainfall over the previous week is the strongest single input, which is hydrologically expected, but the terrain and location features collectively carry more of the model's gain (0.523) than rainfall does (0.367). That is the quantitative form of the component's core claim: knowing *where* the camp is - how high, how close to the channel, how it compares with surrounding terrain - matters as much as knowing how hard it has rained. The single-feature ablation adds an important qualification: those location features are individually redundant, so removing any one of them barely moves the score. An implementation that dropped to rainfall alone would lose only about 0.006 ROC-AUC, but that figure understates the group contribution because the ablation never removes the group together.

The third interpretation is negative and is treated as a finding in its own right. Two scenarios behaved in a physically implausible way - a high gauge reading with little rain scored 0.025, and heavy rain on its own scored 0.116 - and the intermediate configuration's scenario suite matched expectation in only 4 of 12 cases. The unifying explanation is a mismatch between what the water-level feature meant during training (a synthetic index derived from rainfall, elevation and proximity) and what it means at inference time (a live gauge reading in metres). Until that contract is aligned, the model should be read as primarily a rainfall-and-terrain model with a

gauge input it has not learned to trust. This is discussed further as a threat to construct validity in Section 7.5.

7.2 Comparison with Literature Baselines and Existing Solutions

Direct numeric comparison with published flood models is methodologically unsafe, and the literature says so itself: data quality, label definitions and sampling conditions vary so widely between studies that reported accuracies are not commensurable [14], and reviews of machine-learning flood prediction report performance ranges spanning several decades of AUC across datasets [24]. Two structural differences make the comparison even less direct here. The published models are fitted to observed gauge or satellite records at basin scale [9, 8], whereas this component is trained on a synthetic national corpus and evaluated at single coordinates. And their target variable is typically a reach or grid cell, whereas the target here is *this location*.

What can be compared is capability, and there the component is positioned against the requirement set rather than against a score. Table 2.1 summarised that comparison: unlike FLOODWALL [7] or the IoT-GIS lowland system [12], this component produces a point-specific statement for an arbitrary, previously uninstrumented coordinate; unlike satellite pipelines it does not depend on cloud-free imagery [10, 11]; and unlike a bare commercial probe [29] it reasons about the surrounding terrain instead of only reporting a local water level. The critical review of Ahmed et al. [6] identified exactly this combination - multiple sensor categories fused with precise geographic location - as missing from the deployed systems it examined, and that is the gap this component was built to close.

Two comparisons also cut against the component. First, process-based and hydrodynamic approaches can simulate inundation extents that a classifier cannot [25]; the trade-off accepted here is interpretability and edge-cost in exchange for physical fidelity. Second, the adaptive warning frameworks in the flood early-warning literature emphasise that a warning is only effective if it is communicated in a form an officer can act on [16] - which is precisely why this component returns a probability, an ordered category and a per-feature reason, and why the presentation

inconsistency between the warning flag and the category band noted in Section 6.2 matters.

7.3 Research Contribution

The research contributions of this thesis are four.

26. **C1 - A point-specific location-aware flood probability model.** A supervised classifier that accepts an arbitrary coordinate with terrain, proximity and rainfall context and returns a calibrated flood probability for that coordinate, addressing the gap identified by [6]. The feature contract, the leakage exclusions and the derived-feature formulas are specified so that the design can be replicated.
27. **C2 - A two-protocol evaluation with independent re-verification.** Model selection and robustness (Protocol A) are reported separately from the tuned deployment configuration (Protocol B), and the deployed artefact is re-scored independently rather than trusted on the strength of its own metadata. This exposed a real discrepancy - recorded recall 0.7065 against 0.8603 on the served artefact - and traced it to calibration, which is exactly the kind of silent divergence that erodes trust in deployed models.
28. **C3 - Quantified contribution and redundancy of location features.** Gain importance and single-feature ablation are reported together, showing that location and terrain features carry the majority of model gain while being individually substitutable - a nuanced result that neither measure would have revealed alone.
29. **C4 - A behavioral test suite that falsified part of the design.** Scenario testing with SHAP attribution exposed the train-versus-inference skew in the water-level feature and quantified it (4 of 12 scenario outcomes correct in the intermediate configuration), turning a latent defect into a documented, reproducible finding for future implementations.

7.4 Product Contribution

The product contribution is a working capability inside the Smart Camp Box: an enrichment service that resolves terrain, river geometry and live gauge context for

any coordinate within explicit timeouts; a versioned model artefact that carries its own feature order, threshold, class distribution and metrics so that any instance can state what produced a given number; a prediction and alert path with de-duplication and store-and-forward publication; and a test suite that covers the prediction surface. Against the requirements in Table 4.2, NFR01 is met with 61.7 ms mean inference against a 30-second budget, NFR02 by local persistence and provider fallback chains, NFR05 by per-provider timeouts, and the predictive half of the component's objectives is met with the results in Chapter 6.

Scalability to the target of 50 devices follows from the architecture rather than from any single component: edge endpoints are stateless, alert topics are namespaced per camp on MQTT, prediction history is written to indexed local tables and synchronised with an explicit critical priority, and no camp's prediction depends on another camp's state. The cost position established in the proposal - a per-unit price of LKR 371,400 against a hardware bill of materials of LKR 181,070 - leaves margin for the software, support and replacement cycle that a government procurement would require (Section 8.3).

7.5 Limitations and Threats to Validity

Threats are classified using the conventional categories: *external* (does the result generalise?), *internal* (was the result caused by what we think?), *construct* (did we measure the right thing?) and *reproducibility* (can it be repeated?). Table 7.1 enumerates them with their status. None invalidates the reported results; several bound how far they may be pushed. No threat was silently mitigated: items marked *report only* were left unchanged in the repository.

Table 7.1. Threats to validity and limitations.

Threat or limitation	Category	Impact	Status
All 25,000 records are synthetic (`is_synthetic = True`); no observed flood events	External	Field performance may differ from reported figures	Dataset documented; conclusions scoped to the corpus
Single dataset, single country, no cross-region evaluation	External	Generalisation unproven	Recorded as future work (Section 9.4)
Decision threshold selected on the held-out test split	Internal	Threshold metrics optimistically biased	Reported in Section 3.3; re-selection on a validation split proposed
Derived features computed on the full	Internal	Mild preprocessing leakage	Reported; effect size not

Threat or limitation	Category	Impact	Status
dataset before splitting (mean elevation, maxima)			quantified
Recorded metrics come from the pre-calibration booster while the calibrated estimator is served	Construct	Two valid but different metric sets for one artefact	Both reported and reconciled in Section 6.1.4
Training-versus-inference skew in `elevation_ratio` (mean elevation in training, gauge minor level at inference) and `water_level_m` (synthetic index versus live gauge)	Construct	Explains the two implausible scenario outcomes	Identified, quantified, not corrected
Optuna study is unseeded	Reproducibility	A fresh search may return different hyperparameters	Reported; seeds proposed
Pickle contains scikit-learn 1.9.0 objects while `requirements.txt` pins 1.4.2	Reproducibility	Emits `InconsistentVersionWarning`; inference verified working	Report only - pin and re-serialise proposed
Scenario suite: 4 of 12 outcomes matched expectation in the intermediate configuration	Construct	Behavioral validity partial	Reported in Sections 6.1.3 and 6.5
Two disaster-domain tests fail from a stale database mock; 19 further failures lie outside this component	Internal	Incomplete regression coverage of alerting	Report only - root cause identified
Latency measured in-process on a development workstation; no end-to-end HTTP or field measurement	External	NFR01 not verified under field conditions	Reported in Section 6.2
No user or field-officer evaluation of the outputs	External	Usability and trust unproven	Project-level interview reported in Section 6.3; deployment study proposed

Two further limitations are qualitative. The label itself is generated rather than observed, so the model has learned the generator's notion of flood-prone, not hydrology; PR-AUC of 0.549 against a 0.099 baseline should be read in that light. And the component's outputs currently end at an MQTT message: whether that message changes a decision depends on interfaces owned by other components, which were verified for contract but not exercised in a live exercise.

7.6 Implications

Technical. Three practices generalise beyond this component: publish the threshold with the metrics; evaluate a deployed artefact by loading and scoring it rather than by trusting its metadata; and treat feature semantics as a contract that must hold identically at training and at inference. The last of these produced the only substantive defect found in this study.

Operational. For disaster-management users, the practical implication is that a camp can be given a quantitative risk statement within minutes of registration, without waiting for a gauge, a station or a survey. The recall-oriented threshold deliberately accepts roughly one false warning for every true one, which is a policy choice that should be made explicitly by the operating agency - it can be re-tuned per district by changing a single configuration value rather than retraining.

Policy. The result supports the localisation agenda of the multi-hazard early-warning framework [4] and the universal coverage goal of the Early Warnings for All initiative [5]: the expensive part of early warning has traditionally been installing and maintaining fixed stations, and a terrain-and-satellite-plus-local-sensor approach reduces the per-site cost of extending coverage to temporary settlements that no agency will ever instrument permanently.

8 COMMERCIALISATION AND SUSTAINABILITY

8.1 Value Proposition and Target Users

Value proposition. For every temporary relief camp, a self-contained device converts its coordinates into a continuously updated, quantified flood-probability statement with an explanation, an alert channel and local persistence that survives connectivity loss. The value is measured in lead time: in the Kolonnawa case, 8 of 25 active camps had to be closed after conditions deteriorated, and the decision rested on visual observation because nothing at the site could say how close the location was to a threshold.

Target users. Primary users are the Disaster Management Centre and divisional secretariats, which coordinate camp activation and closure; camp managers and field officers, who act on the alert; and non-governmental organisations operating camps, which need the same evidence for their own duty of care. Secondary users are the Irrigation Department and researchers, who gain a distributed observation network that reports terrain-referenced water conditions at points where no gauge exists.

8.2 Market and Competitive Position

The competitive frame has three categories. Commercial water-level probes - for example the AWARE flood IoT range [29] - are reliable at the installation point but require a dedicated instrumented site, so extending coverage to 300 temporary camps means 300 installations, calibration visits and maintenance contracts. Basin-scale forecasting systems [7, 8] deliver high value at low site counts but cannot answer a question about an ungauged location. Low-cost academic nodes [30, 22] occupy the middle but rarely ship with calibration, explanation and alert de-duplication included.

This component competes on *coverage per site cost* rather than on sensor accuracy: any coordinate can be scored because the terrain context comes from open elevation and hydrography data rather than from site survey, and the site still functions when the network does not. For a government agency facing hundreds of short-lived sites, that is the differentiating economic property. Barriers to entry are modest - the technique is reproducible - so the durable advantage is the integrated device, the

operational workflow and the accumulated deployment data rather than the model alone.

8.3 Business or Sustainability Model

The proposal establishes the cost basis reproduced in Table 8.1. Each unit carries a hardware bill of materials of LKR 181,070 and is offered to government at LKR 371,400, which embeds assembly, configuration and margin; the disaster-monitoring software component of which this thesis forms part was budgeted at LKR 65,000 of development cost; annual operating cost for the programme is estimated at LKR 394,240; and national coverage of 300 camps is costed at LKR 111,420,000.

Table 8.1. Cost summary for the programme (LKR).

Item	Amount (LKR)	Note
Hardware bill of materials per unit	181,070	ESP32 bridge, Raspberry Pi 5, 4G modem, sensors, solar and battery, enclosure
Software development - disaster monitoring component	65,000	Covers monitoring, prediction and alerting (this component is part of this line)
Price per Smart Camp Box to government	371,400	Includes configuration, commissioning and margin
Annual operating cost (programme)	394,240	Connectivity, replacement and maintenance
National deployment, 300 camps	111,420,000	Full-coverage scenario

Sustainability. Four properties keep the system running after the research phase. Energetically, the unit is designed around a 100 W solar panel and battery backup, so it does not depend on camp grid power. Technically, the stack is open source with pinned dependencies and scripted tests, which makes maintenance possible without a proprietary vendor. Operationally, offline buffering means the device earns its keep even when connectivity does not, and the model can be retrained and re-issued as a versioned artefact without touching the firmware. Economically, the margin between bill of materials and unit price funds replacements - the item that determines real service life in field conditions.

8.4 Scalability Intellectual Property and Compliance

Scalability. The design target of 50 devices per deployment (NFR03) is met structurally: edge inference is stateless, MQTT topics are namespaced by camp, each

device persists and synchronises its own history with critical priority, and the cloud backend mirrors predictions into PostgreSQL for the dashboard. Moving from 50 to 300 devices is an infrastructure scaling question - broker capacity, database throughput and provider rate limits - not an architectural one.

Intellectual property. The project claims no patents. Protectable assets are the repository, the trained artefacts and their metadata format, the feature contract, and the accumulated operational data; the open-source components (XGBoost, scikit-learn, FastAPI, Express, MQTT) are used under their respective licences. Attribution obligations from open geographic data - chiefly OpenStreetMap for hydrography - must be preserved in any public release.

Compliance. The component processes no personal data: coordinates describe monitored sites, not individuals, so data-protection exposure is limited to the location of camps, which is already published by the authorities. Transport security, token authentication and secret management follow the platform's existing conventions (NFR04). For public procurement, the reproducibility artefacts in Section 3.7 - pinned dependencies, scripted tests, versioned model metadata - are the evidence base for technical evaluation.

8.5 Contribution to the Sustainable Development Goals

The component contributes to two goals directly. **SDG 11**, to make cities and human settlements inclusive, safe, resilient and sustainable [39], is addressed by reducing loss of life and displacement in temporary settlements through earlier, location-specific warning. **SDG 13**, to take urgent action on climate change and its impacts [40], is addressed by building adaptive capacity: the same device that reports today's risk becomes the local observation layer that documents intensifying rainfall extremes over time. Indirectly the work supports the Early Warnings for All objective of protecting every person from hazard-related disasters by 2027 [5], because its cost structure is what makes coverage of uninstrumented, temporary settlements feasible at all.

9 CONCLUSION AND FUTURE DIRECTIONS

9.1 Conclusion

This thesis set out to determine whether a self-contained field device could convert a GNSS fix, local environmental observations and external hydrological data into a reliable, explainable flood probability for the exact location of a relief camp. The answer is a qualified yes. The implemented component reaches 0.9530 ROC-AUC and 0.9094 accuracy on held-out data after independent re-verification, returns a calibrated probability with an explanation in about 62 ms, survives provider and network failure through explicit timeouts and local persistence, and passes every test that covers its prediction surface. The location-aware claim holds quantitatively: terrain and location features carry 0.523 of the model's gain against 0.367 for rainfall.

The qualified part matters equally. Performance was established on a synthetic corpus rather than observed floods; the threshold was chosen on the test split; the feature contract drifts between training and inference in two places, which produced two physically implausible scenario outcomes; and no field officer has yet used the output to make a decision. These are not footnotes - they bound the claims, and they are enumerated in Table 7.1 so that the next iteration inherits them as a work list rather than rediscovering them as defects.

9.2 Achievement of Objectives

Table 9.1 summarizes the status of the five individual objectives against the evidence presented in this thesis.

Table 9.1. Achievement of individual objectives.

Objective	Status	Evidence	Residual qualification
IO1 - Location and elevation profiling	Achieved	GNSS fix path; DEM elevation provider; Table 5.2 timeouts; `elevation_m`, `latitude`, `longitude` in the feature vector	Elevation falls back to GNSS altitude if both providers fail
IO2 - Environmental observation	Achieved	Rainfall and DHT11 modules; `/api/rainfall` and `/api/dht11` integration points in Table 5.3	Implemented with DHT11 rather than the DHT22 specified in the proposal
IO3 - Geospatial water-path analysis	Achieved	Overpass nearest-river query; `distance_to_river_m`, `river_proximity_score` and `drainage_index` derived features	Provider timeout of 35 s can omit the river distance under load
IO4 - External data	Achieved	Gauge, weather and elevation providers with	Gauge input is under-used

Objective	Status	Evidence	Residual qualification
integration		fallback chains; Table 6.10 inputs demonstrating live gauge use	by the model (Sections 6.1.3 and 7.1)
IO5 - Prediction, explanation and alerting	Achieved	Protocol B results in Tables 6.5 to 6.7; SHAP explanations; `flood_predictions` persistence; de-duplicated MQTT alerts	Two alert-evaluator tests fail from a stale mock; no field exercise

9.3 Individual Research and Product Contributions

The product contribution is a working, tested capability inside the Smart Camp Box: location-aware enrichment with explicit timeouts and provider fallback, a versioned and self-describing model artefact, a recall-oriented prediction and alert path with de-duplication and store-and-forward delivery, and regression coverage of the prediction surface. The research contributions are the point-specific formulation of the problem that the critical review of [6] identified as missing, the two-protocol evaluation with independent re-verification that surfaced a calibration-driven metric divergence, the combined gain-importance and ablation analysis showing location features to be individually redundant but collectively dominant, and the behavioral scenario suite that falsified part of the feature contract. Together they advance the component from a proof of concept to a characterised, bounded and honestly reported artefact.

9.4 Future Directions

The following work follows directly from the findings, in priority order.

30. **F1 - Repair the feature contract.** Compute `elevation_ratio` and `water_level_m` with identical formulas at training and at inference, retrain, and re-run the scenario suite; the 4-of-12 scenario result is the acceptance test for this change.
31. **F2 - Re-tune the threshold out of sample.** Select the threshold on a validation split and report test metrics once, removing the optimism identified in Table 7.1, and re-examine whether 0.30 or the F1-optimal 0.35 is preferable under an explicit cost model.
32. **F3 - Evaluate on observed data.** Replace or augment the synthetic corpus with recorded gauge and rainfall events, ideally including the Kolonnawa floods, and add cross-region validation to establish external validity.

33. **F4 - Group-wise ablation and feature clustering.** Remove location features as a group and cluster correlated inputs to test the redundancy hypothesis directly, which the single-feature ablation could not resolve.
34. **F5 - Field deployment study.** Instrument two or three camps, measure end-to-end latency and alert delivery on the Raspberry Pi 5, and observe officers using the probability, category and explanation - the receptive validation missing from Section 6.3.
35. **F6 - Engineering hardening.** Re-serialise the artefact under the pinned scikit-learn version, seed the Optuna sampler, align the warning flag with the category bands, and fix the stale database mock so the disaster suite returns 26 of 26.
36. **F7 - Extend the hazard scope.** Introduce a landslide target once labelled ground-movement data exists, fulfilling the remaining half of the proposal's hazard mandate with the same location-aware machinery.

REFERENCES

[1] United Nations Sri Lanka, "Cyclone Ditwah Situation Report – 23 January 2026," 2026. [Online]. Available: https://srilanka.un.org/en/308868-cyclone-ditwah-situation-report-23-january-2026

[2] United Nations Development Programme, "One-Fifth of Sri Lanka Inundated by Cyclone Ditwah," 2025. [Online]. Available: https://www.undp.org/asia-pacific/press-releases/one-fifth-sri-lanka-inundated-cyclone-ditwah-undp-analysis

[3] Sri Lanka Disaster Management Centre, "DMC Functions and Mandate," 2023. [Online]. Available: https://www.dmc.gov.lk/index.php?option=com_content&view=article&id=72&Itemid=234&lang=en

[4] UNDRR and WMO, "Global Status of Multi-Hazard Early Warning Systems 2025," 2025. [Online]. Available: https://www.undrr.org/reports/global-status-mhews-2025

[5] United Nations, "Early Warnings for All Initiative," 2025. [Online]. Available: https://www.un.org/en/climatechange/early-warnings-for-all

[6] A. Ahmed et al., "Role of the Internet of Things in Flood Risk Management: A Critical Review on Current Practices and Future Directions," *Natural Hazards*, 2025.

[7] C. Prakash, A. Barthwal, and D. Acharya, "FLOODWALL: A Real-Time Flash Flood Monitoring and Forecasting System Using IoT," *IEEE Sensors Journal*, vol. 23, no. 1, pp. 787–799, 2023.

[8] M. Khalaf et al., "IoT-Enabled Flood Severity Prediction via Ensemble Machine Learning Models," *IEEE Access*, vol. 8, pp. 70375–70386, 2020.

[9] Y. Choden et al., "Real-Time Flood Forecasting in Amo Chhu Using Machine Learning and Internet of Things," *Cogent Engineering*, vol. 11, no. 1, 2024.

[10] L. Hashemi-Beni et al., "Mapping Floods from Remote Sensing Data and Quantifying the Effects of Surface Obstruction by Clouds and Vegetation," *Remote Sensing of Environment*, 2023.

[11] R. Munawar et al., "Remote Sensing Methods for Flood Prediction: A Review," *Sensors*, 2022.

[12] S. Kuznichenko et al., "Development of an IoT-Based Flood Monitoring System Integrated with GIS for Lowland Agricultural Areas," *Sensors*, vol. 25, no. 17, 2025.

[13] N. Hamoodi et al., "Integrating Machine Learning and Geospatial Data Analysis for Comprehensive Flood Hazard Assessment," *Environmental Science and Pollution Research*, 2024.

[14] T. Mosavi et al., "Data Quality and Uncertainty Issues in Flood Prediction: A Systematic Review," *International Journal of Digital Earth*, 2025. [Online]. Available: https://www.tandfonline.com/doi/full/10.1080/17538947.2025.2495738

[15] S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in *Advances in Neural Information Processing Systems 30*, 2017, pp. 4765–4774.

[16] J. Castillo-Riquelme et al., "Designing Effective Flood Early Warning Systems: A Narrative Review," *Journal of Flood Risk Management*, 2025. [Online]. Available: https://repository.library.noaa.gov/view/noaa/72059/noaa_72059_DS1.pdf

[17] Sri Lanka Disaster Management Centre, "Early Warning Systems," 2023. [Online]. Available: https://www.dmc.gov.lk/index.php?option=com_content&view=article&id=60&Itemid=225&lang=en

[18] Sri Lanka Irrigation Department, "Hydrology and Disaster Management Division," 2023. [Online]. Available: https://www.irrigation.gov.lk/web/index.php?option=com_content&view=article&id=27&Itemid=128&lang=en

[19] A. V. Pahuriray et al., "IoT-Enabled Flood Monitoring and Early Warning Systems," *International Journal of Computer Science and Mobile Computing*, vol. 14, no. 4, pp. 50–67, 2025. [Online]. Available: https://www.ijcsmc.com/docs/papers/April2025/V14I4202515.pdf

[20] M. Siddique, T. Ahmed, and M. S. Husain, "Flood Monitoring and Early Warning Systems – An IoT Based Perspective," *EAI Endorsed Transactions on Internet of Things*, vol. 9, no. 2, 2023.

[21] I. Ahmed et al., "An Internet of Things and AI-Powered Framework for Long-Term Flood Risk Evaluation," *IEEE Internet of Things Journal*, vol. 11, pp. 3812–3819, 2023.

[22] Z. Zakaria et al., "Development of a Smart Sensing Unit for LoRa WAN-Based IoT Flood Monitoring and Warning System," *Internet of Things and Cyber-Physical Systems*, vol. 2, pp. 212–225, 2022.

[23] M. Islam et al., "Evolution from Physical Process-Based Approaches to Machine Learning Approaches to Predicting Urban Floods: A Literature Review," 2024. [Online]. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC12276141/

[24] H. Mosavi et al., "Flood Prediction Using Machine Learning Models: Literature Review," *Water*, vol. 10, no. 11, p. 1536, 2018.

[25] X. Li et al., "Large-Scale High-Resolution Hydrodynamic Modelling of Urban Floods: Some Practical Considerations," *Environmental Modelling & Software*, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S1570644325000085

[26] Y. Liu et al., "Landslide Prediction Based on Low-Cost and Sustainable Early Warning Systems with IoT," *Bulletin of Engineering Geology and the Environment*, vol. 82, 2023.

[27] M. Ragnoli et al., "Wireless Sensor Network-Based Rockfall and Landslide Monitoring Systems: A Review," *Sensors*, 2023.

[28] H. Thirugnanam et al., “Smart Monitoring of Landslide Systems in Changing Climate: Intelligent Approaches and Future Perspectives,” 2025. [Online]. Available: https://www.intechopen.com/online-first/1226663

[29] AWARE Monitoring Systems, “AWARE Flood IoT Sensors,” 2025. [Online]. Available: https://www.awaremonitoringsystems.com/products/aware-flood-iot

[30] E. Skoubris and G. Hloupis, “An AI-Powered, Low-Cost IoT Node Oriented to Flood Early Warning Systems,” *Engineering Proceedings*, vol. 58, no. 1, p. 44, 2023.

[31] B. Tian et al., “A Review of Cutting-Edge Sensor Technologies for Improved Flood Monitoring and Damage Assessment,” *Sensors*, 2024.

[32] S. Samadi, “The Convergence of AI, IoT, and Big Data for Advancing Flood Analytics Research,” *Frontiers in Water*, vol. 4, 2022.

[33] K. Peffers et al., “A Design Science Research Methodology for Information Systems Research,” *Journal of Management Information Systems*, vol. 24, no. 3, pp. 45–77, 2007.

[34] A. R. Hevner et al., “Design Science in Information Systems Research,” *MIS Quarterly*, vol. 28, no. 1, pp. 75–105, 2004.

[35] T. Akiba et al., “Optuna: A Next-Generation Hyperparameter Optimization Framework,” in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2019, pp. 2623–2631.

[36] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[37] T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[38] L. Breiman, “Random Forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[39] United Nations DESA, “Goal 11: Make Cities Inclusive, Safe, Resilient and Sustainable,” 2025. [Online]. Available: https://sdgs.un.org/goals/goal11

[40] United Nations DESA, “Goal 13: Take Urgent Action to Combat Climate Change,” 2025. [Online]. Available: https://sdgs.un.org/goals/goal13

GLOSSARY

Terms are defined once here and used with these meanings throughout the thesis.

Ablation study. Re-evaluating a model after removing one input to measure that input's contribution.

Alert threshold. The probability at which a warning is issued; 0.30 in the deployed model, deliberately below 0.50 to favour recall.

Brier score. Mean squared error of probabilistic forecasts; lower is better calibrated.

Calibrated probability. A raw model score transformed (here, isotonic regression) so that it can be read as the empirical likelihood of the event.

Class imbalance. A dataset in which one class is much rarer than the other; 9.9 per cent positive here.

Cross-validation. Repeated stratified k-fold splitting used to estimate performance variance without touching the test set.

Digital elevation model (DEM). A raster of terrain heights; used to resolve a site's elevation for any coordinate.

Drainage index. A normalized descriptor of how readily water drains from a location.

Edge gateway. The Raspberry Pi hosted service that enriches, infers and persists data before anything leaves the device.

False negative rate. The share of flood-positive cases missed by the model; the primary safety error.

Feature contract. The agreement between training and inference about what a feature means; broken for two features in the current implementation.

Gain importance. The average improvement a feature contributes when it is used to split a tree, normalized to sum to one.

GNSS. Global Navigation Satellite System; the source of the site's latitude, longitude and altitude.

Hydro spatial. The combination of a coordinate with terrain, hydrography and gauge context, produced by the enrichment service.

Isotonic regression. A non-parametric calibration mapping applied by `CalibratedClassifierCV` to make scores behave like probabilities.

Leakage. Any input that would not be available at prediction time but correlates with the label, inflating apparent performance.

NMEA 0183. The standard sentence format emitted by GNSS receivers and parsed at the edge.

PR-AUC. Area under the precision-recall curve; the informative discrimination measure under class imbalance.

Risk category. The ordered band (Low, Medium, High) assigned from fixed probability bands (below 0.40, 0.40-0.70, 0.70 and above).

ROC-AUC. Area under the receiver operating characteristic curve; threshold-independent discrimination ability.

SHAP. SHapley Additive exPlanations; a per-prediction attribution of each feature's contribution to the output.

Stratified split. A train-test split that preserves the class ratio in both parts.

Water-level index. The synthetic `water_level_m` feature combining normalized rainfall, inverse elevation and river proximity, scaled to a 0-8 range.

APPENDICES

Appendix A - Extended Results

A.1 Full dataset schema. The training corpus contains 32 columns: `record_id`, `district`, `place_name`, `latitude`, `longitude`, `elevation_m`, `distance_to_river_m`, `landcover`, `soil_type`, `water_supply`, `electricity`, `road_quality`, `population_density_per_km2`, `built_up_percent`, `urban_rural`, `rainfall_7d_mm`, `monthly_rainfall_mm`, `drainage_index`, `ndvi`, `ndwi`, `water_presence_flag`, `historical_flood_count`, `infrastructure_score`, `nearest_hospital_km`, `nearest_evac_km`, `flood_risk_score`, `flood_occurrence_current_event`, `inundation_area_sqm`, `is_good_to_live`, `reason_not_good_to_live`, `is_synthetic`, `generation_date`. Of these, ten are model inputs, four are declared leakage exclusions, and the remainder are contextual or administrative fields.

A.2 Full threshold sweep of the eight-feature intermediate configuration. The complete 12-point sweep from Figure 6.8 is reproduced in Table A.1; the deployed configuration's own sweep is given in Section 6.1.5.

Table A.1. Full threshold sweep (5,000 records).

Threshold	Accuracy	Precision	Recall	F1	F2	FNR	FPR	TP	FP	FN	TN
0.15	0.8068	0.2794	0.6053	0.3824	0.4908	0.3947	0.1711	299	771	195	3735
0.20	0.8444	0.3107	0.4717	0.3746	0.4274	0.5283	0.1147	233	517	261	3989
0.25	0.8678	0.3468	0.3826	0.3638	0.3749	0.6174	0.0790	189	356	305	4150
0.30	0.8828	0.3827	0.3036	0.3386	0.3167	0.6964	0.0537	150	242	344	4264
0.35	0.8878	0.3816	0.2186	0.2780	0.2390	0.7814	0.0388	108	175	386	4331
0.40	0.8938	0.4011	0.1518	0.2203	0.1734	0.8482	0.0249	75	112	419	4394
0.45	0.8952	0.3790	0.0951	0.1521	0.1119	0.9049	0.0171	47	77	447	4429
0.50	0.8968	0.3690	0.0628	0.1073	0.0752	0.9372	0.0118	31	53	463	4453
0.55	0.8982	0.3529	0.0364	0.0661	0.0444	0.9636	0.0073	18	33	476	4473
0.60	0.9008	0.4615	0.0243	0.0462	0.0300	0.9757	0.0031	12	14	482	4492
0.65	0.9016	0.6000	0.0121	0.0238	0.0151	0.9879	0.0009	6	4	488	4502
0.70	0.9012	0.5000	0.0040	0.0080	0.0051	0.9960	0.0004	2	2	492	4504

A.3 Scenario suite outcomes for the intermediate configuration. Table A.2 reproduces the 12 scored scenarios whose overall accuracy of 4 of 12 is reported in Section 6.5.

Table A.2. Scenario outcomes: predicted versus expected.

Scenario	Probability	Predicted	Expected	Match
Normal conditions	0.000	low	low	Yes
Heavy rainfall	0.178	low	medium	No
High water level	0.000	low	high	No
Low elevation + rain	0.368	low	high	No
High elevation + rain	0.071	low	low	Yes
Near river + high water	0.209	low	high	No
Distant from river	0.036	low	low	Yes
Flash flood scenario	0.593	medium	high	No
Drought conditions	0.000	low	low	Yes
Coastal flooding	0.190	low	medium	No
Extreme rainfall + low elevation	0.634	medium	high	No
Moderate rain + very close to river	0.228	low	high	No

A.4 Failing-test diagnostic. The two disaster-domain failures raise the following exception, reproduced verbatim from the Vitest output:

```
TypeError: disasterDb.exec is not a function
  at ensureTable (services/disaster/camp-status-store.js:8:14)
  at getAllCampStatusOverrides (services/disaster/camp-status-store.js:36:3)
  at getCampsData (services/disaster/disaster.js:13:21)
  at getCampById (services/disaster/disaster.js:25:17)
  at buildAlert (services/disaster/alert-evaluator.js:23:16)
  at evaluateFloodPrediction (services/disaster/alert-evaluator.js:68:17)
  at tests/disaster/alert-evaluator.test.js:114:37
```

The mock of `config/database.js` supplied by `tests/disaster/alert-evaluator.test.js` implements `prepare` only, while `camp-status-store.js` legitimately calls `exec` during table initialisation. The production database object provides both methods; the defect is confined to the test fixture.

Appendix B - Academic Integrity and AI Use Disclosure

This appendix is provided in accordance with the module policy on generative artificial intelligence, which requires disclosure of any AI assistance while leaving responsibility with the candidate.

Tools used. A large-language-model coding assistant was used during the preparation & idea gathering of this thesis and the supporting analysis scripts.

Work performed by the candidate personally. The candidate selected the research questions, the two-protocol evaluation design and every interpretation, comparison and conclusion in this thesis; ran the training, evaluation and test commands; designed the scenario suite; interviewed the stakeholder during the proposal stage; and takes responsibility for the originality of the claims. All quantitative results were verified by re-executing the repository's own scripts and, where the artefact's own metadata was involved, by independent re-scoring as reported in Section 6.1.4.

Verification. Every number quoted in this thesis is traceable to a repository artefact: the dataset CSV, the two validation report JSON files, the model pickle metadata, the reproduced metric summaries, or a test-suite run executed for this document. Figures were generated from those artefacts by a script retained with the submission.

Declaration. I declare that the use of AI assistance has been disclosed above, that it has been used in accordance with the module policy, and that I remain fully responsible for the accuracy, originality, citations, code, data, analyses and conclusions presented in this thesis.

Signature: _____  ___ Date: 2026/10/11